

RFSOC 4X2 FPGA IMPLEMENTATION ANSPECT O-EMA™ J.83B TX/RX

[NDA-TITLE]

by

Sathwik Sudhindranath

APPROVED BY SUPERVISORY COMMITTEE:

Dr. Benjamin Carrion Schaefer, Chair

[MEMBER 2 PENDING]

[MEMBER 3 PENDING]

Copyright © 2026

Sathwik Sudhindranath

All rights reserved

RFSOC 4X2 FPGA IMPLEMENTATION ANSPECT O-EMA™ J.83B TX/RX

[NDA-TITLE]

by

SATHWIK SUDHINDRANATH, B.E., Electronics & Communication Engineering,
Visvesvaraya Technological University, 2021

THESIS

Presented to the Faculty of
The University of Texas at Dallas
in Partial Fulfillment
of the Requirements
for the Degree of

MASTER OF SCIENCE IN
ELECTRICAL ENGINEERING

THE UNIVERSITY OF TEXAS AT DALLAS

December 2026

ACKNOWLEDGMENTS

December 2026

RFSOC 4X2 FPGA IMPLEMENTATION ANSPECT O-EMA™ J.83B TX/RX

[NDA-TITLE]

Sathwik Sudhindranath, MS
The University of Texas at Dallas, 2026

Supervising Professor: Dr. Benjamin Carrion Schaefer, Chair

TABLE OF CONTENTS

ACKNOWLEDGMENTS	iv
ABSTRACT	v
LIST OF FIGURES	xi
LIST OF TABLES	xii
List of Abbreviations	xii
LIST OF ABBREVIATIONS	xiii
CHAPTER 1 INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Contributions	1
1.4 Thesis Organization	1
CHAPTER 2 FPGA AND RFSOC BACKGROUND	2
2.1 FPGA Architecture Overview	2
2.1.1 Configurable Logic: LUTs, FFs, DSP Slices, and BRAM	2
2.1.2 FPGA vs. ASIC: Why FPGA for Prototyping	5
2.2 Zynq UltraScale+ RFSoc 4x2	7
2.2.1 Processing System: ARM Cortex-A53	9
2.2.2 Programmable Logic: UltraScale+ Fabric	10
2.2.3 RF Data Converter: DAC/ADC Architecture	10
2.2.4 Hardened SD-FEC Cores	15
2.2.5 PS-PL Integration Model	15
2.3 AXI Interconnect Ecosystem	16
2.3.1 AXI4-Stream: Data Path Protocol	17
2.3.2 AXI4-Lite: Control/Register Interface	18
2.3.3 AXI DMA: Memory-Mapped to Stream Bridge	18
2.4 RF Clocking Architecture	19
2.4.1 Clock Requirements for Multi-GHz DAC/ADC	19
2.4.2 LMK04828: Jitter Cleaner and Clock Distributor	21

2.4.3	LMX2594: RF Frequency Synthesizers	22
2.4.4	xrfclk Software API	23
2.5	PYNQ Framework	24
2.5.1	Overlay Concept: .bit + .hwh + Python Driver	24
2.5.2	PYNQ DMA and IP Drivers	25
2.5.3	Why PYNQ for RFSoc Development	26
2.6	Vivado Design Flow	27
2.6.1	IP Integrator Block Design	28
2.6.2	IP Packaging: Custom RTL Modules	29
2.6.3	Synthesis, Implementation, and Bitstream Generation	29
2.7	MATLAB-to-RTL Workflow	31
2.7.1	HDL Coder: Intended Path	31
2.7.2	HDL Coder Limitations Encountered	32
2.7.3	Decision: Manual RTL Coding	33
2.7.4	Fixed-Point Design Considerations	34
CHAPTER 3	DSP ALGORITHM BACKGROUND	36
3.1	DOCSIS 3.0 Downstream Overview	37
3.2	J.83B Annex B Standard	38
3.3	Reed-Solomon Encoding	40
3.3.1	Code Definition	40
3.3.2	Encoding Procedure	41
3.3.3	Extended Parity Symbol	41
3.4	Convolutional Interleaving	42
3.4.1	Forney Interleaver Structure	43
3.4.2	Operating Levels and Mode Selection	43
3.4.3	Parameters Used in This Work	45
3.5	Randomization	45
3.5.1	PN Sequence Generator	46
3.5.2	Initialization and Frame Synchronization	46

3.5.3	Randomizer Parameters	47
3.6	Frame Sync Encoding	47
3.6.1	FEC Frame Structure	47
3.6.2	Sync Trailer Format	48
3.6.3	Symbol Serialization	49
3.7	Trellis Coding	49
3.7.1	Trellis Group Structure	50
3.7.2	Binary Convolutional Coder	51
3.7.3	Differential Precoder	51
3.7.4	Overall Code Rate and Output	52
3.8	256-QAM Symbol Mapping	52
3.8.1	Constellation Geometry	53
3.8.2	C-Word to Symbol Mapping	53
3.8.3	IQ Output and Symbol Rate	54
3.9	SRRC Pulse Shaping	54
3.9.1	Nyquist Matched Filter Pair	55
3.9.2	J.83B Filter Specification	56
3.9.3	RF Output Signal Model	56
CHAPTER 4	IMPLEMENTATION	58
4.1	System Architecture Overview	58
4.2	Processing System	59
4.2.1	UDP/GigE RX	59
4.2.2	TX Chain	59
4.2.3	DMA Control	59
4.3	Programmable Logic: TX Chain	60
4.3.1	PL-01: AXI DMA MM2S	60
4.3.2	PL-02: Transport Framing	60
4.3.3	PL-03: Reed-Solomon Encoder	60
4.3.4	PL-04: Convolutional Interleaver	60

4.3.5	PL-05: Randomizer	60
4.3.6	PL-06: Frame Sync Encoder	60
4.3.7	PL-07: Trellis Encoder	60
4.3.8	PL-08: QAM-256 Mapper	60
4.3.9	PL-09: SRRC FIR TX	60
4.3.10	PL-10: SD-FEC LDPC	60
4.3.11	PL-11: Register File RTL	60
4.3.12	PL-12: IQ Combiner	60
4.4	Programmable Logic: RX Chain	61
4.4.1	PL-13: SRRC FIR RX	61
4.4.2	PL-14: Symbol Timing Recovery	61
4.4.3	PL-15: Carrier and Phase Recovery	61
4.4.4	PL-16: QAM-256 Demapper	61
4.4.5	PL-17: Trellis Decoder	61
4.4.6	PL-18: Frame Sync Detection	61
4.4.7	PL-19: De-Randomizer	61
4.4.8	PL-20: De-Interleaver	61
4.4.9	PL-21: Reed-Solomon Decoder	61
4.4.10	PL-22: Transport Framing RX	61
4.4.11	PL-23: AXI DMA S2MM	61
4.5	RF Data Converter	62
4.5.1	DAC: Tile 228	62
4.5.2	ADC: Tile 224	62
4.6	RF Front-End	63
4.6.1	TX Path	63
4.6.2	RX Path	63
4.7	Clocking Subsystem	64
4.7.1	LMK04828	64
4.7.2	LMX2594	64

4.7.3	xrfclk API	64
4.8	Test and Measurement	65
4.8.1	H-Thunder-1 Reference Signal	65
4.8.2	HDHomeRun: Source and Sink	65
4.8.3	Spectrum Analysis	65
4.9	Verification and Results	66
4.9.1	Test Vectors	66
4.9.2	Loopback Validation	66
CHAPTER 5 CONCLUSION AND FUTURE WORK		67
5.1	Summary of Contributions	67
5.2	Future Work	67
BIOGRAPHICAL SKETCH		68
CURRICULUM VITAE		

LIST OF FIGURES

2.1	Block diagram of the RFSoc 4x2 board, showing the principal hardware subsystems of the AMD Zynq UltraScale+ ZU48DR device [?].	8
2.2	RFDC tile map of the AMD Zynq UltraScale+ ZU48DR device on the RFSoc 4x2 board, showing all Radio Frequency (RF)-Analog-to-Digital Converter (ADC) and RF-Digital-to-Analog Converter (DAC) tiles, their block indices, board-assigned labels, and SubMiniature version A (RF coaxial connector) (SMA) routing status [? ?].	13
3.1	J.83B Annex B downstream processing chain [?].	36
3.2	J.83B Annex B convolutional interleaver structure [?]. Branch b introduces a delay of $b \cdot J$ symbols; Branch 1 is a direct connection with zero delay. Input commutator cycles at the Reed-Solomon (RS) symbol rate; output feeds the randomizer.	44

LIST OF TABLES

2.1	UltraScale+ programmable logic resource types and roles in the J.83B implementation [? ? ?].	5
2.2	Available logic resources of the AMD Zynq UltraScale+ XCZU48DR device [?].	11
2.3	ZU48DR RF data converter parameters (RFSoc 4x2 board) [? ?].	14
2.4	RFDC tile and block map for the AMD Zynq UltraScale+ ZU48DR on the RFSoc 4x2 board [? ?].	14
2.5	ZU48DR Processing System (PS)-Programmable Logic (PL) interface summary [? ?].	16
3.1	DOCSIS 3.0 downstream SC-QAM channel parameters (North America) [? ?].	37
3.2	J.83B Annex B downstream processing chain stages [?].	38
3.3	J.83B Reed-Solomon block structure [?].	42
3.4	J.83B Reed-Solomon code parameters [?].	42
3.5	J.83B Level 2 interleaver parameter options [?].	44
3.6	J.83B Annex B randomizer parameters for 256-QAM operation [?].	47
3.7	J.83B Annex B 256-QAM Forward Error Correction (FEC) frame arithmetic [?].	48
3.8	J.83B Annex B 256-QAM FEC frame sync trailer structure [?].	48
3.9	J.83B Annex B trellis coding parameters for 256-QAM operation [?].	53
3.10	J.83B Annex B 256-QAM symbol mapping parameters [?].	54
3.11	J.83B Annex B Square-Root Raised Cosine (SRRC) pulse shaping parameters for 256-QAM operation (standard-sourced) [?].	57

LIST OF ABBREVIATIONS

ADC	Analog-to-Digital Converter
AMBA	Advanced Microcontroller Bus Architecture
API	Application Programming Interface
APU	Application Processing Unit
ARM	Advanced RISC Machine
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
BCC	Binary Convolutional Code
BRAM	Block Random-Access Memory
CLB	Configurable Logic Block
CMTS	Cable Modem Termination System
CPE	Customer Premises Equipment
CPU	Central Processing Unit
CL	Configurable Logic
DAC	Digital-to-Analog Converter
DDC	Digital Down-Converter
DDR	Double Data Rate
DMA	Direct Memory Access

DOCSIS	Data Over Cable Service Interface Specification
DSP	Digital Signal Processing
DUC	Digital Up-Converter
EVM	Error Vector Magnitude
FEC	Forward Error Correction
FF	Flip-Flop
FIFO	First-In First-Out
FIR	Finite Impulse Response
FPGA	Field-Programmable Gate Array
GSPS	Giga-Samples Per Second
HFC	Hybrid Fiber-Coaxial
HP	High-Performance
HDL	Hardware Description Language
IP	Intellectual Property
IQ	In-Phase/Quadrature
ISI	Inter-Symbol Interference
ITU-T	International Telecommunication Union – Telecommunication Standardization Sector
LDPC	Low-Density Parity-Check

LFSR	Linear Feedback Shift Register
LSB	Least Significant Bit
LUT	Look-Up Table
LUT6	Six-Input Look-Up Table
LED	Light-Emitting Diode
MAC	Multiply-Accumulate
MM	Memory-Mapped
MM2S	Memory-Mapped to Stream
MPEG-TS	MPEG Transport Stream
MSB	Most Significant Bit
MPSoC	Multi-Processor System-on-Chip
ML	Machine Learning
NCO	Numerically Controlled Oscillator
NRE	Non-Recurring Engineering
OS	Operating System
PL	Programmable Logic
PN	Pseudo-Noise
PS	Processing System
QAM	Quadrature Amplitude Modulation

256-QAM	256-Point Quadrature Amplitude Modulation
RF	Radio Frequency
RFDC	RF Data Converter
RFSoc	Radio Frequency System-on-Chip
RPU	Real-Time Processing Unit
RS	Reed-Solomon
RTL	Register-Transfer Level
RAM	Random-Access Memory
S2MM	Stream to Memory-Mapped
SC-QAM	Single-Carrier Quadrature Amplitude Modulation
SD-FEC	Soft-Decision Forward Error Correction
SG	Scatter-Gather
SNR	Signal-to-Noise Ratio
SRRC	Square-Root Raised Cosine
SDMA	Streaming Direct Memory Access
SPI	Serial Peripheral Interface
SMA	SubMiniature version A (RF coaxial connector)
TCM	Trellis Coded Modulation
Tcl	Tool Command Language

VCO	Voltage-Controlled Oscillator
VCXO	Voltage-Controlled Crystal Oscillator
VHDL	VHSIC Hardware Description Language
XML	Extensible Markup Language
IP-XACT	IP eXtensible Architecture for Component Technology

CHAPTER 1

INTRODUCTION

- 1.1 Motivation
- 1.2 Problem Statement
- 1.3 Contributions
- 1.4 Thesis Organization

CHAPTER 2

FPGA AND RFSOC BACKGROUND

2.1 FPGA Architecture Overview

Field-Programmable Gate Arrays (FPGAs) are reconfigurable integrated circuits whose internal logic connections and functional behavior are defined by a configuration bitstream loaded into the device at runtime rather than fixed during fabrication [?]. This programmability distinguishes FPGAs from Application-Specific Integrated Circuits (ASICs), which are fabricated for a single immutable function, and from general-purpose processors, which execute sequential instruction streams in software. The FPGA occupies a distinct position in the design space between these two extremes: it offers near-hardware throughput through massively parallel datapath execution while retaining the ability to reprogram the device to modify or extend functionality without incurring a new fabrication cycle [?].

Modern FPGAs have evolved from simple programmable logic devices of the mid-1980s into heterogeneous system-on-chip platforms that integrate reconfigurable fabric, hardened processing cores, high-speed memory, and, in the most recent generations, directly integrated radio-frequency data converters on a single die [?]. The Xilinx Zynq UltraScale+ family, and specifically the Radio Frequency System-on-Chip (RFSoc) 4x2 board used in this work, represents this convergence [?] of programmable logic with an Advanced RISC Machine (ARM) processing system and monolithically integrated RF data converters. The sections that follow describe the fundamental building blocks of the UltraScale+ programmable logic fabric and establish the rationale for selecting an FPGA-based platform for implementing the J.83B digital signal processing chain.

2.1.1 Configurable Logic: LUTs, FFs, DSP Slices, and BRAM

The programmable logic of a Zynq UltraScale+ FPGA is organized around four principal resource types: Look-Up Tables (LUTs), Flip-Flops (FFs), hardened Digital Signal Processing

(DSP) slices, and Block Random-Access Memory (BRAM). Each resource type serves a distinct role in implementing a digital signal processing pipeline, and their relative utilization governs the achievable throughput, clock frequency, and resource efficiency of a given design [? ?].

Look-Up Tables: A LUT is a small programmable truth table that implements any combinational logic function of up to six binary inputs [?]. In the UltraScale+ architecture, eight Six-Input Look-Up Table (LUT6) primitives are grouped with dedicated carry logic into a Configurable Logic Block (CLB), the fundamental tiling unit of the reconfigurable fabric [?]. Beyond combinational logic, a LUT may be configured as a small distributed RAM element or a shift-register delay line, providing additional flexibility for shallow buffering without consuming dedicated BRAM resources [?]. In the J.83B processing chain implemented in this work, LUTs realize the finite-state machines governing the trellis encoder and the convolutional interleaver commutator, the Pseudo-Noise (PN) sequence comparison logic of the randomizer, and the 256-QAM symbol look-up table that maps 8-bit C-words to complex constellation coordinates.

Flip-Flops: Each LUT primitive in the UltraScale+ CLB is paired with one or more FF registers that capture and hold logic values on the rising edge of the clock [?]. Flip-flops serve as pipeline stage registers throughout a design, breaking long combinational paths into shorter segments and enabling the fabric to sustain higher clock frequencies [?]. In a pipelined DSP datapath such as the J.83B transmit and receive chain, FFs insert registered boundaries between each processing block to permit sustained throughput at the 250 MHz PL clock frequency available on the RFSoc 4x2 platform [?]. The pipeline depth, measured in clock cycles, determines the end-to-end latency through the chain, but does not reduce the steady-state symbol throughput once the pipeline is filled.

DSP48E2 Slices: The DSP48E2 is a hardened arithmetic primitive embedded throughout the UltraScale+ fabric that provides a 27×18 -bit two's-complement multiplier cascaded with a 48-bit accumulator [?]. A dedicated cascade port allows adjacent DSP48E2 slices to be chained together without consuming general routing resources, making the primitive well suited to Multiply-Accumulate (MAC)-intensive operations such as Finite Impulse Response (FIR) filtering [?]. The UltraScale+ family provides approximately 12,000 DSP48E2 slices capable of operating at approximately 900 MHz, representing a fourfold increase in DSP bandwidth compared to the preceding 28 nm generation [?]. In the J.83B transmit chain, the SRRC pulse shaping filter is a 121-tap complex FIR kernel whose multiply-accumulate operations map directly onto DSP48E2 slices, leveraging the cascade path to achieve the required throughput at the 250 MHz fabric clock.

Block RAM: BRAM is a dedicated on-chip memory primitive available in 36 Kbit units, each configurable as a true dual-port memory that supports simultaneous read and write access on independent ports at the full fabric clock rate [?]. The deterministic, single-cycle read latency of BRAM makes it suitable for real-time streaming data paths where pipeline-regulated access timing is required [?]. In the J.83B implementation, BRAM is used to implement the shift-register delay lines of the Forney convolutional interleaver, which requires $I(I - 1)J = 16,256$ symbol positions of storage for the $I = 128$, $J = 1$ mode mandated by the standard [?], as well as packet buffers in the Advanced eXtensible Interface (AXI) DMA data path between the processing system and the programmable logic.

The four resource types together define the programmable logic capacity available for implementing the complete J.83B processing chain. Table 2.1 summarizes each resource type and its primary role in this work.

Table 2.1. UltraScale+ programmable logic resource types and roles in the J.83B implementation [? ? ?].

Resource	Primitive	Role in This Work
LUT6	6-input programmable truth table [?]	Combinational logic: FSMs, QAM symbol look-up table, control paths
FF	D flip-flop, clocked register [?]	Pipeline staging; enables 250 MHz PL clock operation
DSP48E2	27×18 multiplier + 48-bit accumulator [?]	SRRC pulse shaping filter MAC operations; cascade path for 121-tap FIR
BRAM	36 Kbit true dual-port RAM [?]	Interleaver delay lines (16,256 symbol positions); AXI DMA packet buffers

2.1.2 FPGA vs. ASIC: Why FPGA for Prototyping

The selection of an FPGA over an ASIC or a software-based processor for implementing the J.83B physical layer is motivated by three interconnected engineering arguments: reconfigurability without fabrication cost, throughput adequate for the target symbol rate, and the practical constraints of prototype-scale development.

Reconfigurability and NRE Cost: An ASIC implementation of the J.83B processing chain would offer lower silicon area, higher clock rates, and lower power dissipation than an equivalent FPGA implementation [?]. Kuon and Rose conducted empirical measurements across 23 benchmark circuits implemented at the 90 nm node in both an FPGA and equivalent standard-cell ASIC technologies, and found that, for designs using only reconfigurable logic fabric, the FPGA silicon area is on average 35 times larger than the ASIC counterpart, with a critical-path delay ratio of approximately 3.4 and a dynamic power consumption ratio of approximately 14 [?]. When designs exploit hardened heterogeneous blocks such as multiplier-accumulators and block memories, the area gap narrows substantially to an average of 18 times, because the FPGA’s dedicated primitives approach the efficiency of their ASIC counterparts for those specific operations [?].

These area, speed, and power penalties are accepted in exchange for two decisive practical advantages. First, an FPGA carries no Non-Recurring Engineering (NRE) cost: unlike an ASIC, which requires a photomask set and a fabrication cycle of several months before any silicon is available for testing [?], an FPGA is an off-the-shelf device programmed by downloading a bitstream in seconds. Design iterations that would require a costly re-spin on an ASIC are accomplished on an FPGA by regenerating and reloading the bitstream, enabling the rapid algorithm-to-hardware iteration cycle that prototype and research implementations demand [?]. For low-volume production, including academic research platforms, the per-unit device cost of an FPGA is manageable without volume to amortize ASIC NRE expenditure [?].

Throughput Advantage Over Software Processors: A software implementation of the J.83B chain on a general-purpose or DSP processor would serialize the eight-stage processing chain, reducing throughput relative to the hardware-parallel FPGA realization. Diouri et al. compared the implementation of a 40th-order bandpass FIR filter on an Altera Cyclone III FPGA and a Texas Instruments TMS320C6713 DSP processor under identical computational objectives [?]. The FPGA implementation achieved an estimated throughput of 3.4 Gops/s at 122 MHz with 28 active multiplier elements, compared to 1.8 Gops/s for the DSP processor operating at 225 MHz [?]. The FPGA power consumption was measured at approximately 95 mW, versus a peak of 900 mW for the DSP processor at full utilization [?]. The throughput advantage of the FPGA arises from its fully parallel datapath: all multiply-accumulate operations within a filter tap execute simultaneously in hardware, whereas the DSP processor executes one multiply-accumulate per instruction cycle [?]. For the J.83B symbol rate of 5.360537 Msym/s [?], the parallel FPGA datapath provides the throughput margin necessary to implement each processing stage as a pipelined hardware module operating within the 250 MHz PL clock budget.

UltraScale+ as a Prototyping Substrate: The Zynq UltraScale+ RFSoc device used in this work integrates a high-performance reconfigurable logic fabric with an ARM Cortex-A53 processing system, approximately 12,000 DSP48E2 slices at 900 MHz, and directly integrated radio-frequency data converters on a single 16 nm FinFET die [?]. The 16 nm FinFET process delivers 60% higher programmable logic performance and 2.5 times greater performance per watt compared to the preceding 28 nm generation, while the fourfold increase in DSP bandwidth enables computationally intensive blocks such as the SRRC pulse shaping filter to be realized entirely within the PL fabric [?]. The monolithic integration of the RF data converters eliminates the external converter interfaces that contribute latency and power overhead in multi-chip architectures [?], and the PYNQ framework provides a Python-based host interface to the programmable logic overlay that accelerates the prototype iteration cycle without requiring a low-level embedded software development environment [?]. Taken together, these properties establish the Zynq UltraScale+ RFSoc as an appropriate platform for prototyping and validating the complete J.83B transmit and receive chain developed in this thesis.

2.2 Zynq UltraScale+ RFSoc 4x2

The hardware platform used in this work is the RFSoc 4x2 development board, produced by Real Digital in partnership with AMD [?]. The board is built around the AMD Zynq UltraScale+ ZU48DR device, a third-generation member of the RFSoc product family that monolithically integrates a multi-core ARM processing system, a high-density UltraScale+ programmable logic fabric, and directly integrated RF-class analog data converters on a single 16 nm FinFET die [? ?].

The integration of analog and digital subsystems on a common die eliminates the external data converter interfaces and interboard interconnects that contribute latency, power overhead, and signal integrity constraints in conventional multi-chip RF signal processing architectures [?]

]. The ZU48DR device therefore provides a compact, single-board platform on which the complete J.83B transmit and receive processing chain can be implemented, verified, and characterized.

The principal subsystems of the ZU48DR relevant to this work are the processing system, the programmable logic fabric, the integrated RF data converter, and the hardened Soft-Decision Forward Error Correction (SD-FEC) cores. Each is described in the sections that follow.



Figure 2.1. Block diagram of the RFSoc 4x2 board, showing the principal hardware subsystems of the AMD Zynq UltraScale+ ZU48DR device [?].

2.2.1 Processing System: ARM Cortex-A53

The PS of the ZU48DR is a heterogeneous multi-processor complex that contains two independent processor clusters operating at distinct privilege and performance levels [? ?].

Application Processing Unit: The Application Processing Unit (APU) consists of four ARM Cortex-A53 cores implementing the 64-bit ARMv8-A instruction set architecture at a clock frequency of up to 1.2 GHz [?]. Each Cortex-A53 core includes a 32 KB level-one instruction cache and a 32 KB level-one data cache with ECC protection, and the four cores share a 1 MB level-two unified cache [?]. The APU supports a 64-bit Double Data Rate (DDR)4 external memory interface with ECC, providing the high-bandwidth system memory required for the Direct Memory Access (DMA) data paths connecting the processing system to the programmable logic [?]. A full-featured Linux environment runs on the APU, and the PYNQ framework exposes the programmable logic overlay to the host through a Python Application Programming Interface (API) that operates within this Linux environment [?].

Real-Time Processing Unit: The Real-Time Processing Unit (RPU) consists of two ARM Cortex-R5F cores implementing the 32-bit ARMv7-R instruction set at up to 600 MHz [? ?]. The Cortex-R5F cores support both lockstep and split-mode operation: in lockstep mode the two cores execute identical instruction streams in parallel for fault detection, while in split mode they execute independent tasks simultaneously [?]. Each core includes a tightly coupled memory interface that provides deterministic, low-latency access to on-chip instruction and data memory without cache miss penalties, making the RPU suitable for real-time control tasks that require bounded execution time [?]. In the implementation described in Chapter 4, the APU under Linux manages all data plane transfers and configuration of the programmable logic; the RPU is not used in this work.

2.2.2 Programmable Logic: UltraScale+ Fabric

The programmable logic fabric of the ZU48DR is an UltraScale+ array whose resource classes were described individually in Section 2.1.1. The fabric provides the configurable logic blocks, DSP48E2 slices, block RAM, and UltraRAM primitives required to implement the J.83B processing pipeline as a collection of pipelined hardware modules [? ?].

The PL fabric operates from a reference clock derived from the PS clock wizard, designated `p1_clk0`, at a frequency of 250 MHz, which drives all AXI4-Stream data path modules and the DMA engines in the implementation [?]. In the theoretical limit of one symbol per clock cycle, the fabric clock would support a maximum symbol throughput of 250 Msym/s, exceeding the 5.360537 Msym/s J.83B symbol rate [?] by a factor of approximately 47. In practice, the achievable throughput is determined by the pipeline architecture of the Register-Transfer Level (RTL) modules instantiated on the PL fabric, which introduces multi-cycle latency stages in the Reed-Solomon encoder, convolutional interleaver, and SRRC pulse shaping filter; the 250 MHz clock domain nonetheless provides ample throughput margin for the 5.360537 Msym/s target rate across all implemented pipeline stages.

The PL fabric communicates with the PS through a set of high-bandwidth AXI interconnect ports described in Section 2.2.5. All RTL modules implementing the J.83B processing stages are packaged as Vivado Intellectual Property (IP) cores with AXI4-Stream data interfaces and AXI4-Lite control interfaces, and are integrated into the block design as described in Section 2.6 [?].

Table 2.2 lists the available logic resource counts for the XCZU48DR device as specified in the product data sheet [?].

2.2.3 RF Data Converter: DAC/ADC Architecture

The defining feature of the RFSoc device class is the monolithic integration of high-speed analog data converters with the digital logic fabric on a single die [?]. The ZU48DR

Table 2.2. Available logic resources of the AMD Zynq UltraScale+ XCZU48DR device [?].

Resource	Count
CLB Look-Up Tables (LUT6)	425,280
CLB Flip-Flops	850,560
DSP48E2 Slices	4,272
Block RAM (36 Kb tiles)	1,080
UltraRAM (288 Kb blocks)	80
Clock Management Tiles (CMT)	8
High-Performance (HP) I/O	299
High-Density (HD) I/O	48
GTY Transceivers	16
SD-FEC Cores (hardened)	8
RF-ADC Channels (total)	8
RF-DAC Channels (total)	8

integrates four RF-ADC tiles and four RF-DAC tiles, each tile containing multiple converter channels clocked by a shared sample clock distributed from an on-chip clock network [?].

RF-ADC: The ZU48DR integrates four RF-ADC tiles (tiles 224–227), each containing two converter channels. Of these, two tiles are routed to board-accessible SMA connectors: Tile 224 provides channels 0 and 1 (RF ADC A and RF ADC B) and Tile 226 provides channels 0 and 1 (RF ADC C and RF ADC D), yielding four accessible 14-bit RF-ADC channels capable of sampling at up to 5 Giga-Samples Per Second (GSPS) with an analog input bandwidth extending to approximately 6 GHz [? ?]. Tiles 225 and 227 are present on the die but are not connected to any board-level RF interface. Each RF-ADC channel incorporates a programmable Digital Down-Converter (DDC) block that performs digital downconversion from the RF sample stream to a complex baseband In-Phase/Quadrature (IQ) signal at a decimated output rate compatible with the PL fabric clock domain [? ?]. The DDC consists of a Numerically Controlled Oscillator (NCO)-driven complex mixer followed by a configurable decimation filter chain; the NCO frequency is programmable over a range

of 0 to approximately 4915 MHz on the RFSoc 4x2 platform, enabling direct digital tuning to any channel within the RF input bandwidth [?]. The decimated IQ output is presented to the PL fabric through a 32-bit AXI4-Stream interface at the fabric clock rate, carrying one complex sample per transfer as a packed 16-bit I and 16-bit Q integer pair [?].

RF-DAC: The ZU48DR integrates four RF-DAC tiles (tiles 228–231), each containing two converter channels. Of these, two blocks are routed to board-accessible SMA connectors: Tile 228 Block 0 (RF DAC A) and Tile 230 Block 0 (RF DAC B), yielding two accessible 14-bit RF-DAC channels capable of operating at up to 9.85 GSPS [?]. Tile 228 Block 1, Tiles 229, 231, and Tile 230 Block 1 are present on the die but are not connected to any board-level RF interface. Each RF-DAC channel incorporates a programmable Digital Up-Converter (DUC) block that performs digital upconversion from a complex baseband IQ input to the RF sample rate, complementing the DDC path of the RF-ADC [?]. The PL fabric delivers baseband IQ samples to the DAC through a 32-bit AXI4-Stream interface, packing the in-phase and quadrature components as 16-bit signed integers per transfer word, and the DUC interpolates and upconverts the sample stream before it is applied to the DAC core [?].

Latency Advantage: The elimination of the external data converter interface is significant for latency-sensitive applications. Javaid et al. measured the loopback latency of a first-generation Zynq UltraScale+ RFSoc RF Data Converter (RFDC) operating at 4 GSPS and found best-case hardware latencies of 108 ns for real signal processing and 246 ns for complex IQ processing at a 250 MHz stream clock [?]. These values represent an 80 to 90 per cent reduction in latency compared to a conventional discrete converter solution using the JESD204 serial interface, which incurs best-case latencies on the order of $1.66\text{ }\mu\text{s}$ [?]. The latency advantage arises from the elimination of the multi-clock-cycle buffering required by the JESD204 framing and lane alignment protocol [?].

The RFDC parameters used in this work are summarized in Table 2.3.

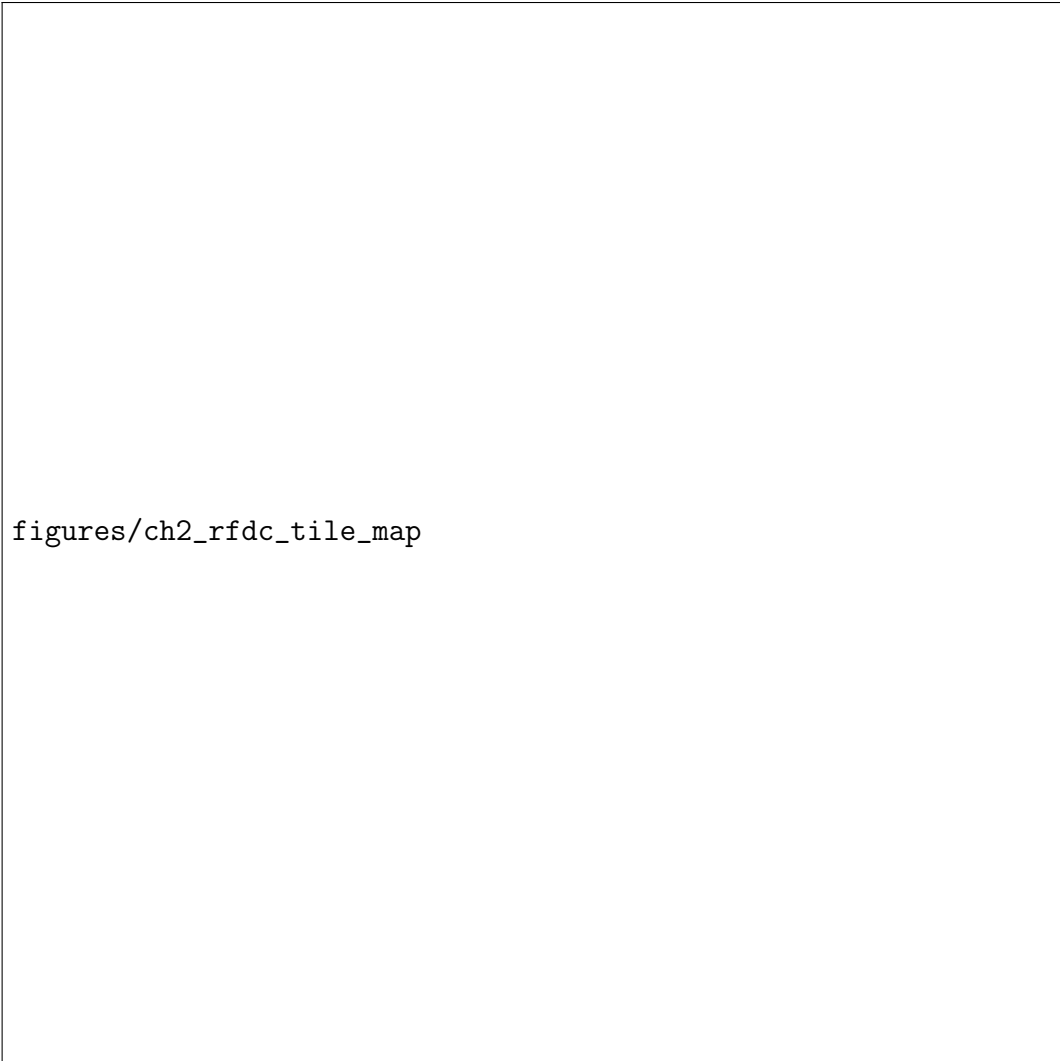


Figure 2.2. RFDC tile map of the AMD Zynq UltraScale+ ZU48DR device on the RFSoc 4x2 board, showing all RF-ADC and RF-DAC tiles, their block indices, board-assigned labels, and SMA routing status [? ?].

Table 2.3. ZU48DR RF data converter parameters (RFSoc 4x2 board) [? ?].

Parameter	RF-ADC	RF-DAC
Channels (accessible)	4	2
Resolution	14 bits	14 bits
Maximum sample rate	5 GSPS	9.85 GSPS
RF bandwidth	≈ 6 GHz	≈ 6 GHz
Digital conversion block	DDC (decimation)	DUC (interpolation)
NCO tuning range	0–4915 MHz	0–4915 MHz
PL interface	AXI4-Stream, 32-bit (16-bit I + 16-bit Q)	AXI4-Stream, 32-bit (16-bit I + 16-bit Q)

Table 2.4 enumerates all twelve converter blocks on the ZU48DR, their tile and block indices, board-assigned labels, and routing status on the RFSoc 4x2 board.

Table 2.4. RFDC tile and block map for the AMD Zynq UltraScale+ ZU48DR on the RFSoc 4x2 board [? ?].

Type	Tile	Block	Board Label	Routed to SMA
RF-DAC	228	0	RF DAC A	Yes
RF-DAC	228	1	—	No
RF-DAC	229	0	—	No
RF-DAC	229	1	—	No
RF-DAC	230	0	RF DAC B	Yes
RF-DAC	230	1	—	No
RF-DAC	231	0	—	No
RF-DAC	231	1	—	No
RF-ADC	224	0	RF ADC A	Yes
RF-ADC	224	1	RF ADC B	Yes
RF-ADC	225	0	—	No
RF-ADC	225	1	—	No
RF-ADC	226	0	RF ADC C	Yes
RF-ADC	226	1	RF ADC D	Yes
RF-ADC	227	0	—	No
RF-ADC	227	1	—	No

2.2.4 Hardened SD-FEC Cores

The ZU48DR integrates eight hardened SD-FEC cores as fixed-function silicon blocks within the device [?]. Unlike PL-based FEC implementations, the hardened SD-FEC cores do not consume any programmable logic resources; they operate as dedicated accelerator blocks connected to the PL fabric through AXI4-Stream data interfaces and an AXI4-Lite control interface [?].

Each SD-FEC core supports both Low-Density Parity-Check (LDPC) and Turbo decoding algorithms, with code parameters configurable at runtime through the control interface [?]. The LDPC mode implements belief-propagation decoding with programmable iteration count and early termination, making it suitable for high-throughput forward error correction in broadband communications applications [?].

In the J.83B processing chain implemented in this work, the SD-FEC cores are used to implement the LDPC encoding and decoding stages of the ANSpect O-EMA platform. Further implementation detail is deferred to Chapter 4 in accordance with the project confidentiality scope.

2.2.5 PS-PL Integration Model

The PS and PL subsystems of the ZU48DR are connected by a set of AXI interconnect ports that form the data and control paths between the ARM processing system and the reconfigurable logic fabric [? ?].

High-Performance Ports: The High-Performance (HP) AXI slave ports connect the PL fabric to the PS memory subsystem at high bandwidth, supporting sustained data rates of up to 85 Gbps per port for read-plus-write traffic [?]. These ports are used by DMA engines in the PL to transfer data buffers between the PL processing pipeline and the DDR4 system memory managed by the APU [?]. In the J.83B implementation, the MM2S channel of the

AXI DMA delivers MPEG Transport Stream (MPEG-TS) packet data from the PS memory to the transmit processing chain, and the S2MM channel transfers demodulated output data from the receive chain back to the PS memory through the HP ports.

General-Purpose Ports: The general-purpose AXI master ports provide the PS with memory-mapped register access to IP cores in the PL, supporting control and status register reads and writes at lower bandwidth but without occupying the HP data path [?]. In this work, the AXI4-Lite register file of each RTL module is connected to the PS through a general-purpose port, allowing runtime configuration of processing parameters from the PYNQ Python environment [?].

Interrupt Routing: The ZU48DR provides a configurable interrupt routing fabric that delivers PL-generated interrupt signals to the PS APU or RPU interrupt controllers [?]. DMA completion interrupts from the PL are routed to the Linux interrupt controller on the APU, enabling the PYNQ DMA driver to block on transfer completion without polling [?].

The PS-PL interface parameters relevant to this implementation are summarized in Table 2.5.

Table 2.5. ZU48DR PS-PL interface summary [? ?].

Port Type	Protocol	Direction	Use in This Work
HP AXI slave	AXI4 Full	PL $\rightarrow$ PS	DMA data transfers (MM2S/S2MM)
GP AXI master	AXI4-Lite	PS $\rightarrow$ PL	Register R/W, runtime config
Interrupt	IRQ	PL $\rightarrow$ PS	DMA completion signal

2.3 AXI Interconnect Ecosystem

The Advanced Microcontroller Bus Architecture (AMBA) AXI4 protocol family is the standard on-chip interconnect used throughout the Zynq UltraScale+ device to connect the processing system, programmable logic IP cores, and integrated peripherals [?]. AMBA

AXI4 defines three distinct sub-protocols, each optimized for a different class of on-chip communication: AXI4-Stream for high-throughput unidirectional data pipelines, AXI4-Lite for low-complexity register access, and AXI4 Full for high-performance Memory-Mapped (MM) burst transfers [?]. Each sub-protocol is described in the sections that follow.

2.3.1 AXI4-Stream: Data Path Protocol

AXI4-Stream is a unidirectional point-to-point streaming protocol that connects a single source to a single sink without address signaling [?]. It carries only a data payload and the handshaking signals needed to regulate the transfer rate between source and sink, making it well suited to deeply pipelined DSP datapaths where data flows in a single direction at a fixed word rate [?].

Signal Interface: The AXI4-Stream interface is defined by four primary signals [?]:

- **TDATA** carries the data payload; its width is a positive integer multiple of 8 bits.
- **TVALID** is driven by the source and asserted when **TDATA** holds a valid transfer.
- **TREADY** is driven by the sink and asserted when the sink is able to accept a transfer.
- **TLAST** is driven by the source and asserted on the final beat of a packet or frame boundary.

A transfer occurs on any rising clock edge on which both **TVALID** and **TREADY** are asserted simultaneously [?]. When the sink cannot accept data it de-asserts **TREADY**, applying backpressure that stalls the source until the transfer can be accepted [?]. This mechanism allows the pipeline to self-regulate without data loss when any stage is temporarily unable to sustain the full transfer rate.

2.3.2 AXI4-Lite: Control/Register Interface

AXI4-Lite is a simplified subset of the full AXI4 protocol designed for low-throughput register read and write transactions [?]. It retains the five-channel handshake structure of AXI4 Full, comprising write address, write data, write response, read address, and read data channels, but restricts transfers to a single beat per transaction at a fixed 32-bit data width aligned to 32-bit word boundaries [?]. These constraints eliminate burst, narrow transfer, and unaligned transfer complexity, producing an interface compact enough to be included in every IP core without significant resource overhead [?].

In the Zynq UltraScale+ PS-PL integration model, the PS acts as the AXI4-Lite master and each PL IP core acts as a slave exposing a word-addressed register file [?]. The Vivado address editor assigns each slave a unique base address in the PS memory map, allowing the ARM Cortex-A53 to access any register in any PL core through ordinary load and store instructions [?].

2.3.3 AXI DMA: Memory-Mapped to Stream Bridge

The AXI DMA is a Xilinx LogiCORE IP block that bridges the MM AXI4 Full interface of the PS memory system and the AXI4-Stream interfaces of the PL fabric [?]. It provides two independent unidirectional transfer channels: the Memory-Mapped to Stream (MM2S) channel reads a contiguous block of data from system memory and presents it as an AXI4-Stream to the PL; the Stream to Memory-Mapped (S2MM) channel accepts an AXI4-Stream from the PL and writes it into system memory [?]. The two channels operate independently and simultaneously, enabling full-duplex data movement between the PS and PL [?].

The AXI DMA supports two operating modes [?]. In simple mode, a single contiguous transfer is initiated by writing the source or destination address and byte count to the DMA control registers via AXI4-Lite; the DMA completes the transfer and raises an interrupt to notify the PS [?]. In Scatter-Gather (SG) mode, a linked list of buffer descriptors stored in

system memory defines a sequence of transfers; the DMA fetches each descriptor, executes the corresponding transfer, and advances to the next descriptor autonomously without PS intervention [?]. SG mode reduces interrupt overhead for multi-packet transfers and enables zero-copy data movement by chaining descriptors that reference existing memory buffers [?].

2.4 RF Clocking Architecture

Accurate, low-noise clocking is a prerequisite for high-order Quadrature Amplitude Modulation (QAM) modulation. In a 256-Point Quadrature Amplitude Modulation (256-QAM) system, each symbol encodes 8 bits on a 16×16 constellation, so the minimum Euclidean distance between adjacent symbols is small relative to the outer constellation radius. Phase noise on the sample clock, the carrier oscillator, or the distribution network rotates symbol clusters in the IQ plane, degrading Error Vector Magnitude (EVM) and raising the probability of symbol error [?] beyond the level correctable by the FEC stack. The RFSoc 4x2 board addresses this requirement through a three-device clocking hierarchy [?]. A Skyworks Si5395 clock jitter attenuator, driven by a 48 MHz crystal reference, generates the primary PS and PL fabric clocks. A Texas Instruments LMK04828 jitter cleaner, supplied with a 10 MHz reference from the Si5395 and a 160 MHz Voltage-Controlled Crystal Oscillator (VCXO) input, produces additional fabric clocks and drives two Texas Instruments LMX2594 RF frequency synthesizers. Each LMX2594 generates the high-speed sample clock reference delivered to the RFDC DAC or ADC tiles [?]. This section describes the clock requirements imposed by the J.83B 256-QAM signal chain and the architecture of each clocking device.

2.4.1 Clock Requirements for Multi-GHz DAC/ADC

The clocking hierarchy of the RFSoc 4x2 platform spans two distinct frequency domains that serve different subsystems and must be treated independently: the RFDC sample clock domain and the PL fabric clock domain.

RFDC Sample Clock Domain: The RFDC DAC and ADC tiles are driven by a shared sample clock that determines the Nyquist bandwidth and timing resolution of the converters [?]. For the J.83B transmit chain implemented in this work, the RFDC output sample rate is set by the product of the J.83B symbol rate and the oversampling factor used in the SRRC pulse shaping filter

$$f_{\text{sample}} = f_{\text{sym}} \times N_{\text{sps}} = 5.360537 \text{ Msym/s} \times 6 = 32.163 \text{ MSamp/s} \quad (2.1)$$

where $N_{\text{sps}} = 6$ is the number of samples per symbol produced by the SRRC filter and $f_{\text{sym}} = 5.360537 \text{ Msym/s}$ is the J.83B Annex B symbol rate [?]. The RFDC tile clock derived from this sample rate drives the DAC core and the interpolation chain of the DUC block within the RFDC [?].

The phase noise and jitter performance of this sample clock directly determine the achievable EVM floor. For a 256-QAM constellation the EVM contribution from sample clock phase noise is approximately [?]

$$\text{EVM}_{\phi} \approx 2\pi f_c \cdot \sigma_{\text{jitter}} \quad (2.2)$$

where f_c is the RF carrier frequency and σ_{jitter} is the root-mean-square integrated jitter of the sample clock over the signal bandwidth. Meeting the EVM requirement of the J.83B 256-QAM channel therefore demands a sample clock source with sufficiently low phase noise that the jitter contribution remains a small fraction of the minimum symbol spacing [?]. The LMK04828 jitter cleaner described in Section 2.4.2 provides this low-noise reference.

PL Fabric Clock Domain: The PL fabric operates in a separate clock domain driven by `p1.clk0`, a 250 MHz clock derived from the PS clock wizard [?]. This clock drives all AXI4-Stream pipeline modules, the AXI DMA engines, and the Streaming Direct Memory Access (SDMA) interfaces connecting the PL to the PS memory subsystem [?]. The 250 MHz

PL clock is entirely independent of the RFDC sample clock: the two domains are separated by AXI4-Stream First-In First-Out (FIFO) crossing stages within the RFDC IP block, which resynchronize the converter sample stream to the PL clock before presenting it to the fabric [?]. The distinction between these two clock domains is consequential for timing closure: all RTL modules in the J.83B processing pipeline are constrained to the 250 MHz PL domain, while the sample-rate timing of the RFDC is governed solely by the external sample clock provided by the LMK04828.

2.4.2 LMK04828: Jitter Cleaner and Clock Distributor

The LMK04828 is a dual-loop clock conditioner and distribution device manufactured by Texas Instruments [?]. On the RFSoc 4x2 board it occupies the second stage of the clocking hierarchy, receiving a 10 MHz reference from the Si5395 on its `CLK_IN1` input and a 160 MHz VCXO signal on its `OSC_IN` input [?].

Dual-Loop Architecture: The LMK04828 implements two independent internal phase-locked loops [?]. The first internal loop is a low-bandwidth reference cleanup loop that accepts the 10 MHz input and phase-locks the on-chip VCXO to it, suppressing high-frequency reference noise within the loop bandwidth [?]. The second internal loop is a high-bandwidth distribution loop that accepts the cleaned VCXO output and drives a high-frequency internal Voltage-Controlled Oscillator (VCO), whose output is divided and routed to multiple programmable output channels simultaneously [?]. The two internal loops are independent; the board provides two status Light-Emitting Diode (LED) indicators, labeled PLL1 and PLL2, that are driven by the LMK04828 `STAT1` and `STAT2` outputs to indicate when each internal loop has achieved lock [?].

Role in This Work: In the RFSoc 4x2 platform used in this thesis, the LMK04828 occupies the intermediate stage of the clocking hierarchy between the Si5395 primary reference and the

LMX2594 RF synthesizers [?]. It provides the cleaned reference signal that the LMX2594 devices require to generate stable, low-jitter sample clocks for the RFDC tiles, and additionally supplies supplementary PL fabric reference clocks to the ZU48DR [?]. The LMK04828 does not drive the RFDC converter cores directly; that function is performed by the downstream LMX2594 synthesizers described in Section 2.4.3.

2.4.3 LMX2594: RF Frequency Synthesizers

The LMX2594 is a wideband fractional- N phase-locked loop synthesizer manufactured by Texas Instruments, capable of generating output frequencies from 10 MHz to 15 GHz [?]. The RFSoc 4x2 board populates two LMX2594 devices: one synthesizer, designated PLL1 on the board, supplies the sample clock reference to the RF-ADC tiles; a second synthesizer, designated PLL2, supplies the sample clock reference to the RF-DAC tiles [?]. Each LMX2594 receives its reference from the LMK04828 output described in Section 2.4.2 and drives a dedicated lock-status LED on the board [?].

Fractional- N PLL Architecture: The LMX2594 employs a fractional- N divider in its feedback path, allowing its output to be set to non-integer multiples of the reference input with fine frequency resolution [?]. The fractional- N architecture incorporates a sigma-delta modulator to randomize the divider switching pattern and suppress spurious tones that would otherwise arise from periodic divider cycling [?]. The synthesizer includes an integrated VCO spanning the full 10 MHz–15 GHz output range without mechanical tuning [?].

Role in This Work: In the RFSoc 4x2 platform, the two LMX2594 devices are the final stage of the clocking chain and are the direct source of the high-frequency sample clock references consumed by the RFDC DAC and ADC tiles [?]. The stability and phase noise of the LMX2594 output therefore set the achievable EVM floor for the J.83B 256-QAM signal processed in this thesis [?]. The RF carrier frequency of the J.83B downstream channel

is not produced by the LMX2594; it is synthesized digitally by the NCO within the RFDC DUC, which is a programmable parameter of the RFDC IP block [?].

2.4.4 `xrfclk` Software API

Both the LMK04828 and LMX2594 are configured through a serial peripheral interface bus that connects them to the ZU48DR processing system [?]. Direct configuration via low-level Serial Peripheral Interface (SPI) register writes from Python would require the host software to maintain device-specific register maps and sequencing rules for each clock synthesizer. The `xrfclk` library, an open-source Python package distributed by Xilinx and maintained as part of the RFSoc software ecosystem, abstracts this low-level access [?].

Library Function: The `xrfclk` library exposes a Python API that accepts a target sample rate and, from it, selects and writes the appropriate pre-computed register sequences to the LMK04828 and LMX2594 over SPI [?]. The register sequences are stored as Python dictionaries within the library, eliminating the need for the host application to implement synthesizer programming logic directly [?]. A single function call from the PYNQ Python environment is therefore sufficient to configure both clock devices to the required operating frequencies before the RFDC overlay is loaded [?].

Integration with PYNQ: The `xrfclk` library is pre-installed in the PYNQ Linux image provided with the RFSoc 4x2 board, and its API is invoked from the same Jupyter notebook environment used to load the PL overlay and drive the DMA data path [?]. The hardware clocking setup therefore integrates naturally into the PYNQ software stack described in Section 2.5, with no separate embedded software development environment required. The specific `xrfclk` function calls and target frequency arguments used in this implementation are deferred to Chapter 4.

2.5 PYNQ Framework

The RFSoc 4x2 board ships with native support for the PYNQ framework, an open-source software environment developed by Xilinx that provides a Python-based interface to the programmable logic of Zynq UltraScale+ devices [?]. PYNQ, an acronym for Python Productivity for Zynq, eliminates the need for embedded C or bare-metal software development to interact with custom hardware in the PL: the designer exposes PL IP cores through a Python driver class and accesses them from a Jupyter notebook running on the Cortex-A53 Linux environment of the PS [? ?]. The framework therefore permits a two-layer design methodology in which hardware-intensive signal processing tasks execute in the PL fabric at full hardware speed, while orchestration, configuration, and data movement are managed from Python [? ?]. The three principal components of the PYNQ programming model are the overlay concept, the DMA and IP driver library, and the Jupyter-based development environment; each is described in the sections that follow.

2.5.1 Overlay Concept: .bit + .hwh + Python Driver

The fundamental abstraction of the PYNQ programming model is the *overlay* [?]. An overlay is a complete hardware configuration for the PL that is packaged as three co-dependent artifacts: a bitstream file (`.bit`), a hardware handoff file (`.hwh`), and a Python driver class [?].

Bitstream: The bitstream (`.bit`) is the binary configuration file produced by the Vivado implementation flow described in Section 2.6.3 [?]. Loading the bitstream programs the UltraScale+ PL fabric with the target hardware design, activating all instantiated IP cores and their AXI interconnect topology [?].

Hardware Handoff File: The hardware handoff file (`.hwh`) is an Extensible Markup Language (XML) description of the block design exported by Vivado at implementation time [?]. It encodes the address map of every PL IP core, the width and connectivity of each AXI4-Lite register file, and the interrupt routing from the PL to the PS [?]. When an overlay is loaded in Python, PYNQ parses the `.hwh` to auto-generate a register-level driver dictionary for every IP core in the design, making the PL hardware immediately accessible from Python without handwritten register definitions [?].

Python Driver Class: The Python driver class instantiates the overlay and provides a high-level API to the underlying hardware [?]. At its simplest, the driver calls `Overlay()` with the `.bit` and `.hwh` paths, which programs the PL and populates an IP dictionary keyed by core name [?]. Designers may extend the base class with domain-specific methods that wrap register reads and writes into meaningful operations, and the PYNQ library provides pre-built driver subclasses for common Xilinx IP cores including the AXI DMA [?].

2.5.2 PYNQ DMA and IP Drivers

The PYNQ library provides built-in driver support for the AXI DMA IP core described in Section 2.3.3, and a general mechanism for accessing the register files of custom PL IP cores [?].

DMA Driver: The `pynq.lib.dma.DMA` class wraps the AXI DMA register interface described in the product guide [?] into a Python object with MM2S and S2MM channel attributes [?]. Each channel exposes a `transfer()` method that accepts a DMA-capable buffer object, writes the buffer address and byte count to the DMA control registers, and optionally blocks until the DMA raises its completion interrupt [?]. The DMA-capable buffer is allocated by the PYNQ `allocate()` function, which reserves a contiguous physically addressed region in the PS DDR4 memory that the PL DMA engine can address directly

without scatter-gather indirection [?]. This allocation strategy ensures that the PL DMA engine reads and writes to a predictable physical address range, avoiding cache coherency complications when transferring data between the PS and PL [?].

IP Register Access: For custom RTL modules that expose an AXI4-Lite register file, PYNQ auto-generates a register accessor from the `.hwh` address map at overlay load time [?]. Each IP core appears as an entry in the overlay’s `ip_dict`, keyed by the instance name assigned in the Vivado block design [?]. Individual registers within each core are accessible by index or offset, allowing the PS Python environment to read status registers and write configuration parameters to any PL module without additional driver development [?].

2.5.3 Why PYNQ for RFSoc Development

The PYNQ framework offers three practical advantages that are particularly well matched to the prototype development cycle of an RFSoc signal processing system.

Rapid Iteration: Modifying a signal processing parameter in a bare-metal C environment requires rebuilding and reflashing the embedded software image, which introduces a compilation and deployment cycle between each test [?]. Under PYNQ, configuration changes are issued as Python assignment statements in a running Jupyter notebook, taking effect on the next AXI4-Lite register write with no recompilation [?]. The Python ecosystem further provides direct access to `numpy` arrays for constructing and inspecting data buffers passed to the PL, and to `matplotlib` for immediate visualization of captured IQ samples or spectral data [?].

Native Board Support: The RFSoc 4x2 board is distributed with a pre-built PYNQ Linux image that includes the `xrfclk` clock configuration library, the RFDC Python driver, and all board support packages required to configure the clocking chain and load a custom

overlay [? ?]. No separate toolchain installation or embedded Operating System (OS) build is required to begin running custom PL designs on the board [? ?].

Academic Adoption: The combination of the PYNQ framework and the RFSoc 4x2 platform has been adopted in published academic signal processing implementations, where the two-layer Hardware Description Language (HDL)/Python architecture allows hardware designers to concentrate on RTL correctness while delegating data movement and test orchestration to Python [?]. The PL processing layer operates at hardware speed, providing throughput that is orders of magnitude higher than the PS alone for compute-intensive tasks [?], while the Python layer retains the flexibility of a high-level scripting environment for test and measurement workflows.

2.6 Vivado Design Flow

AMD Vivado™ design suite is the primary FPGA development environment for UltraScale+ devices and is the toolchain used in this work to integrate the J.83B RTL modules, connect them to the PS, and generate the bitstream that programs the ZU48DR PL fabric [? ?]. Vivado supports a graphical block design environment in which IP cores are instantiated and connected visually, a command-line Tool Command Language (Tcl) scripting interface for reproducible design automation, and a synthesis and implementation engine that maps the integrated design onto the physical resources of the target device [?]. The output of the Vivado flow is a bitstream file (`.bit`) and a hardware handoff file (`.hwh`) that together constitute the PL overlay consumed by the PYNQ framework described in Section 2.5 [?]. The three stages of the Vivado design flow relevant to this work are described in the sections that follow.

2.6.1 IP Integrator Block Design

The Vivado IP Integrator is a graphical design canvas in which IP cores are instantiated as blocks and connected by drawing wires between their ports [?]. The resulting block design is a self-contained description of the PL hardware that captures the AXI interconnect topology, the clock and reset routing, and the PS-PL port bindings in a single artifact [?].

Zynq PS IP: The PS subsystem of the ZU48DR is represented in the block design by the Zynq UltraScale+ Multi-Processor System-on-Chip (MPSoC) IP core, which generates the PS hardware configuration [?]. A graphical configuration dialog controls which PS peripherals are enabled, the DDR4 memory interface parameters, the PL clock outputs, and the mapping of HP and general-purpose AXI ports to the PL interconnect [?]. Exporting the block design from this IP core produces the hardware platform file consumed by downstream Vitis or PYNQ flows [?].

AXI SmartConnect: The AMD SmartConnect IP core manages the AXI interconnect between the PS master ports and the slave register interfaces of the PL IP cores [?]. It performs clock crossing, data-width conversion, and arbitration automatically, allowing IP cores operating in different AXI configurations to be connected without manual bridging logic [?].

Address Editor: The Vivado address editor assigns each AXI4-Lite slave in the block design a unique base address within the PS memory map [?]. These addresses are exported to the `.hwh` file at implementation, where the PYNQ overlay parser reads them to construct the Python IP dictionary described in Section 2.5.1 [?]. Every custom RTL module that exposes an AXI4-Lite register file receives a unique base address in this editor, making its registers directly addressable by the ARM Cortex-A53 [?].

2.6.2 IP Packaging: Custom RTL Modules

Before a custom RTL module can be instantiated in the Vivado IP Integrator block design, it must be packaged as a Vivado IP core using the Vivado IP Packager [?]. The packager wraps a SystemVerilog RTL source hierarchy in a standardized IP-XACT descriptor that the block design canvas can discover, instantiate, and connect [?].

AXI Port Inference: The Vivado IP Packager infers AXI interface types from the port naming conventions of the RTL module [?]. Ports whose names follow the AXI4-Stream prefix convention (`s_axis_*` for slave, `m_axis_*` for master) are automatically grouped into typed AXI4-Stream interface objects that the block design canvas can connect with a single wire, rather than requiring individual port-to-port connections [?]. Similarly, AXI4-Lite register file ports are inferred from the `s_axi_*` naming convention [?].

component.xml: The output of the packaging step is a `component.xml` file that defines the IP core interface, its port groupings, bus interface protocols, and any user-visible parameters exposed through the block design configuration dialog [?]. The packaged core is stored in the Vivado IP repository and is available for instantiation in any block design that references that repository path [?]. The packaging approach therefore supports a modular design methodology in which each RTL stage of the J.83B processing chain is developed, verified, and packaged independently before integration into the top-level block design [?].

2.6.3 Synthesis, Implementation, and Bitstream Generation

Following block design completion, the Vivado flow proceeds through three sequential compilation stages that transform the RTL and block design sources into a device-specific binary configuration: synthesis, implementation, and bitstream generation [?].

Synthesis: Synthesis translates the RTL source files and the block design wrapper into a technology-mapped netlist expressed in terms of UltraScale+ primitives such as LUT6, FF, DSP48E2, and BRAM [?]. The synthesizer performs combinational optimization, register retiming, and resource inference, converting inferred arithmetic operations into DSP48E2 instances and inferred memories into BRAM or distributed Random-Access Memory (RAM) primitives as appropriate [?]. The synthesis report lists resource utilization estimates and identifies any RTL constructs that could not be mapped to available primitives [?].

Implementation: Implementation places the synthesized netlist onto the physical sites of the ZU48DR die and routes the interconnect between placed primitives [?]. The placer assigns each LUT, FF, DSP48E2, and BRAM instance to a specific site on the Configurable Logic (CL) fabric, subject to timing, power, and congestion constraints [?]. The router then connects the placed instances through the UltraScale+ programmable interconnect, and the timing analyzer verifies that every combinational path in the design meets its setup and hold constraints at the target clock frequency [?]. A design that fails timing closure must be modified through pipeline stage insertion, constraint relaxation, or RTL restructuring and re-implemented until all paths meet their constraints [?].

Bitstream Generation: Once implementation achieves timing closure, the Vivado bitstream generator converts the placed and routed netlist into the binary `.bit` file that programs the UltraScale+ configuration memory [?]. Vivado simultaneously exports the hardware handoff (`.hwh`) file from the block design, which records the IP address map and interface metadata used by the PYNQ overlay loader [?]. The `.bit` and `.hwh` files together constitute the complete PL overlay deliverable of this thesis [? ?].

2.7 MATLAB-to-RTL Workflow

The J.83B signal processing chain implemented in this thesis was first developed as a MATLAB reference model, a collection of floating-point scripts that implement each processing stage in software and generate test vectors against which the hardware implementation is subsequently verified. A natural extension of this model-based development approach is to use MathWorks HDL Coder to automatically generate synthesizable RTL directly from the MATLAB or Simulink representation of each processing block, bypassing the manual coding step [?]. This section describes the intended HDL Coder workflow, the specific limitations that were encountered during its evaluation for this project, the engineering decision that resulted from that evaluation, and the fixed-point design considerations that apply to the RTL implementation regardless of the generation method.

2.7.1 HDL Coder: Intended Path

MathWorks HDL Coder is a toolbox that generates synthesizable VHSIC Hardware Description Language (VHDL) or Verilog RTL from MATLAB functions and Simulink models [?]. The generated RTL is intended to be directly importable into an FPGA vendor toolchain such as Vivado, eliminating the need to manually translate an algorithm from a floating-point software model into a fixed-point hardware description [?].

Specified Workflow: ANSpect FPGA specification identified HDL Coder as the primary RTL generation method for five of the eight J.83B processing blocks: the SRRC pulse shaping filter, the randomizer, the trellis encoder, the 256-QAM symbol mapper, and the RS encoder. The intended workflow for each block was: develop and verify a MATLAB or Simulink model of the block against the J.83B standard; apply HDL Coder to generate a fixed-point SystemVerilog or VHDL netlist; package the generated RTL as a Vivado IP core per the flow described in Section 2.6.2; and integrate the packaged core into the top-level block design [?].

]. The remaining three blocks, namely the convolutional interleaver, the frame sync encoder, and the transport framing stage, were identified as candidates for manual RTL coding due to their state-machine-intensive control structures.

Expected Advantages: The model-based approach was expected to reduce RTL coding effort, shorten the iteration cycle between algorithm refinement and hardware testing, and provide a built-in correspondence between the MATLAB reference model and the generated RTL [?]. HDL Coder’s fixed-point conversion wizard was expected to automate the selection of word lengths and scaling factors for each signal in the datapath, replacing a manual fixed-point design step [?].

2.7.2 HDL Coder Limitations Encountered

Evaluation of HDL Coder against the five specified blocks revealed three categories of limitation that prevented the generated RTL from meeting the integration requirements of the Vivado block design flow described in Section 2.6.

AXI4-Stream Interface Generation: The Vivado IP Integrator block design requires each PL module to expose a native AXI4-Stream data interface with compliant **TVALID**, **TREADY**, and **TLAST** handshaking as described in Section 2.3.1. HDL Coder does not natively generate AXI4-Stream interfaces; it generates a data-valid handshake without **TREADY** backpressure support [?]. Adapting the generated RTL to a fully compliant AXI4-Stream interface requires a manually written wrapper that absorbs or propagates backpressure, negating the code-generation advantage for any block that must participate in a streaming pipeline with variable-rate stages.

AXI4-Lite Register File: HDL Coder does not generate AXI4-Lite register files for runtime configuration of processing parameters [?]. Every block in the J.83B implementation

that requires runtime parameter access, including the interleaver depth selector, the randomizer initialization trigger, and the frame sync detector threshold, requires a hand-written AXI4-Lite register file, making the HDL Coder output a partial solution that still requires substantial manual RTL work to become a complete Vivado IP core [?].

Core Datapath Output Only: HDL Coder generates RTL for the core combinational or pipelined arithmetic datapath of a block but does not generate the surrounding infrastructure required for integration into a Vivado IP core [?]. The generated output omits the finite-state machine that sequences data through multi-cycle operations, the control logic that manages pipeline enable, flush, and stall conditions, the AXI4-Stream handshake wrapper that connects the datapath to the streaming pipeline, and the AXI4-Lite register file that exposes runtime configuration parameters to the PS [?]. Each of these components must be written manually and integrated around the generated datapath core before the block constitutes a complete, integration-ready Vivado IP core. For the J.83B processing blocks, the control logic and state machine complexity was comparable to the datapath complexity, so the net RTL coding effort saved by HDL Coder was marginal relative to a fully manual implementation.

2.7.3 Decision: Manual RTL Coding

The three limitation categories described in Section 2.7.2 collectively established that HDL Coder could not produce integration-ready RTL for the J.83B processing blocks without a manual effort comparable to writing the RTL from scratch. The engineering decision was therefore made to abandon the HDL Coder path and implement all PL modules as hand-coded SystemVerilog.

RTL Language and Interface Standard: All J.83B processing blocks and their supporting infrastructure except Vivado IP are implemented in SystemVerilog, with native

AXI4-Stream data interfaces and AXI4-Lite control interfaces coded directly to the AMBA specification described in Section 2.3. Each module is self-contained, fully pipelined, and packaged as a Vivado IP core per the flow described in Section 2.6.2.

Retained Role of MATLAB: The MATLAB reference model is retained in the manual RTL path as a verification tool rather than a code generation source. The reference scripts generate bit-exact test vectors for each processing stage; these vectors are applied to the SystemVerilog modules in simulation and the outputs are compared against the MATLAB reference to confirm algorithmic correctness before the design is taken to hardware. This test-vector methodology provides a functional equivalence check between the floating-point MATLAB algorithm and the fixed-point RTL implementation that is independent of any code generation tool.

Methodology Contribution: The evaluation of HDL Coder against the J.83B processing chain, the documentation of its limitations in the context of AXI4-Stream pipeline integration, and the transition to manual SystemVerilog constitute a validated engineering methodology contribution of this thesis. The decision is grounded in the specific interface requirements of the Vivado IP Integrator flow, providing a documented rationale that is transferable to similar RFSoc development projects. Implementation detail for each RTL module is deferred to Chapter 4.

2.7.4 Fixed-Point Design Considerations

All arithmetic in the J.83B RTL processing chain is implemented in fixed-point representation, since the UltraScale+ PL fabric does not provide floating-point hardware primitives at the throughput required by the 5.360537 Msym/s symbol rate [?]. Fixed-point arithmetic represents each signal as a B -bit signed integer scaled by a power-of-two factor, trading the

dynamic range of floating-point for a deterministic, hardware-efficient representation whose word length maps directly onto LUT, FF, and DSP48E2 primitive widths [?].

Word Length and Scaling: A fixed-point signal is characterized by its total word length B and the number of fractional bits F , giving a resolution of 2^{-F} and a representable range of $[-2^{B-F-1}, 2^{B-F-1} - 2^{-F}]$ for two's-complement signed formats [?]. Increasing B reduces quantization noise at the cost of wider datapath primitives; for DSP operations the critical constraint is that intermediate products fit within the 27×18 -bit input and 48-bit accumulator of the DSP48E2 slice without overflow [?]. Each processing stage in the J.83B chain requires individual word length selection to balance quantization noise against resource utilization.

IQ Data Format: The complex IQ samples produced by the 256-QAM symbol mapper and carried through the SRRC pulse shaping filter are represented as 16-bit signed integers with no fractional bits (`int16` / `fix16_0`), providing a dynamic range sufficient to represent the $\{-15, -13, \dots, +15\}$ amplitude levels of the 256-QAM constellation with headroom for the SRRC filter impulse response [?]. The 32-bit AXI4-Stream word carrying each complex sample packs the 16-bit I component in the lower half and the 16-bit Q component in the upper half, matching the RFDC IP stream format described in Section 2.2.3 [?].

Galois Field Arithmetic: The RS encoder and randomizer operate over $\text{GF}(2^7)$, where all arithmetic is performed modulo the primitive polynomial $p(x) = x^7 + x^3 + 1$ as specified in the J.83B standard [?]. Galois field arithmetic does not use conventional fixed-point scaling: all elements are 7-bit integers and multiplication is defined by the field reduction rule, not by binary multiplication followed by shift [?]. This structural incompatibility with the HDL Coder output model described in Section 2.7.2 reinforced the decision to implement these blocks in manually coded SystemVerilog.

CHAPTER 3

DSP ALGORITHM BACKGROUND

This chapter presents the signal processing background underlying the J.83B Annex B downstream physical layer standard, which forms the algorithmic foundation of the transmit and receive chains implemented in this work. The J.83B Annex B standard, defined by the International Telecommunication Union [?] and adopted as the North American cable modulation standard by ANSI/SCTE [?], specifies a fixed processing chain that transforms an MPEG-TS byte stream into a modulated 256-QAM radio-frequency signal suitable for distribution over Hybrid Fiber-Coaxial (HFC) networks.

The processing chain, illustrated in Figure 3.1, consists of eight sequential stages: transport framing, RS encoding, convolutional interleaving, randomization, frame sync encoding, trellis coding, 256-QAM symbol mapping, and SRRC pulse shaping. Each stage is treated in a dedicated section of this chapter. Implementation details specific to the RFSoc 4x2 realization of each block are deferred to Chapter 4.

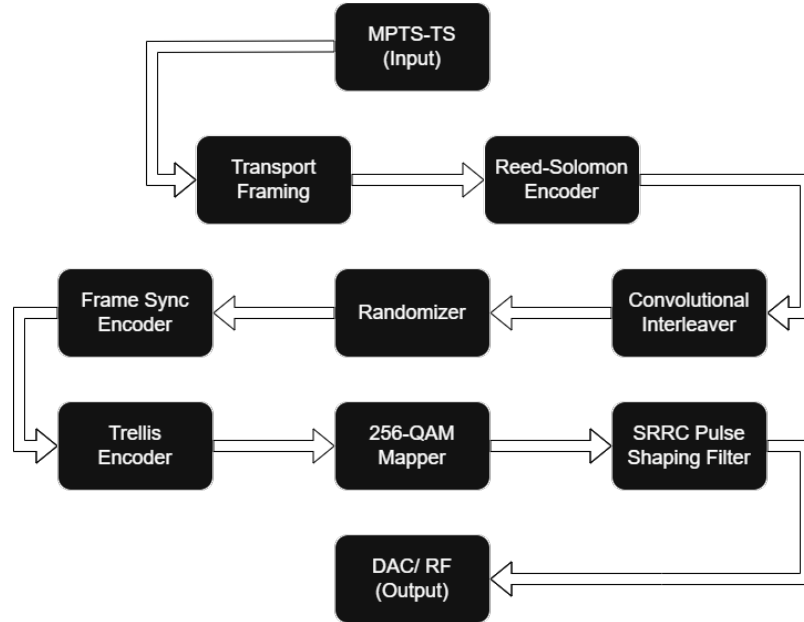


Figure 3.1. J.83B Annex B downstream processing chain [?].

3.1 DOCSIS 3.0 Downstream Overview

Data Over Cable Service Interface Specification (DOCSIS) 3.0 is the cable industry standard governing broadband data transmission over HFC networks [?]. In the downstream direction, from cable headend to Customer Premises Equipment (CPE), DOCSIS 3.0 employs Single-Carrier Quadrature Amplitude Modulation (SC-QAM) channels. Each channel occupies a 6 MHz bandwidth slot in the North American channel plan [?].

The downstream channel structure relevant to this work is summarized in Table 3.1.

Table 3.1. DOCSIS 3.0 downstream SC-QAM channel parameters (North America) [? ?].

Parameter	Value
Channel bandwidth	6 MHz
Modulation	256-QAM (J.83B Annex B)
Symbol rate	5.360537 Msym/s
Bits per symbol	8
Information bit rate	38.81070 Mbps
FEC standard	ITU-T J.83 Annex B

A DOCSIS 3.0 downstream system bonds multiple SC-QAM channels to increase aggregate throughput [?]. Each individual channel is independently modulated and passes through the complete J.83B Annex B physical layer processing chain described in this chapter. The headend QAM modulator accepts a stream of MPEG-TS packets from the Cable Modem Termination System (CMTS) and produces a modulated RF signal for distribution over the coaxial plant. At the subscriber end, a cable modem or set-top box tuner demodulates the RF signal and recovers the MPEG-TS byte stream [?].

The 256-QAM modulation format mandated by J.83B Annex B [?] places stringent requirements on the linearity and phase noise performance of the transmit and receive RF chains, and on the precision of the digital baseband processing blocks described in the sections that follow [?].

3.2 J.83B Annex B Standard

ITU-T Recommendation J.83 defines digital multi-programme systems for television, sound, and data services over cable distribution networks [?]. Annex B of the recommendation specifies the physical layer standard adopted in North America, covering the downstream transmission chain from MPEG transport stream input to the modulated RF output delivered over the HFC plant [? ?].

The Annex B standard defines a fixed, ordered sequence of processing stages applied to each MPEG-TS byte stream before modulation. Table 3.2 lists each stage in order, together with the corresponding standard section reference and its role in the chain.

Table 3.2. J.83B Annex B downstream processing chain stages [?].

Stage	Block	Standard Section	Role
1	Transport Framing	§B.4	MPEG-TS packet alignment
2	Reed-Solomon Encoding	§B.5.1	Forward error correction
3	Convolutional Interleaving	§B.5.2	Burst error dispersal
4	Randomization	§B.5.4	Spectral whitening
5	Frame Sync Encoding	§B.5.3	Receiver frame lock
6	Trellis Coding	§B.5.5	Coded modulation
7	256-QAM Mapping	§B.5.5.5	Symbol constellation
8	SRRC Pulse Shaping	§B.6.1	Bandlimited waveform

Each stage in the chain is described individually in Sections 3.3 through 3.9. The remainder of this section provides an overview of the complete chain and the key system-level parameters that govern its operation.

Transport Framing: The chain begins with transport framing, which operates on MPEG-TS packets of 188 bytes each [?]. Each packet begins with the standard sync byte 0x47 [?]. The framing stage replaces this sync byte in every packet with a computed parity checksum, derived as the coset of an FEC parity check over the preceding 187 payload bytes [?]. The receiver uses the checksum to detect packet boundaries and identify packet errors [?].

Reed-Solomon Encoding: RS encoding provides the primary FEC layer [?]. The encoder operates on blocks of 122 seven-bit symbols and appends 6 parity symbols to produce a 128-symbol output block [?]. The code is defined over the Galois field $GF(2^7)$ and can correct up to 3 symbol errors per block [?].

Convolutional Interleaving: Convolutional interleaving disperses burst errors across multiple Reed-Solomon blocks so that a single burst affecting several consecutive bytes at the receiver is spread into correctable single-byte errors after de-interleaving [?]. The interleaver is a Forney (convolutional) type with depth $I = 128$ and increment $J = 1$.

Randomization: The randomizer applies a PN scrambling sequence to the interleaved byte stream [?]. The scrambler operates in $GF(128)$, initializing its state to `0x7F` at each frame boundary and XOR-ing each 7-bit symbol with the PN sequence output. Randomization prevents long runs of identical symbols that would cause spectral lines in the transmitted signal.

Frame Sync Encoding: Frame sync encoding appends a 40-bit trailer to the end of each FEC frame [?]. The trailer is packed MSB-first into 7-bit symbols and provides the receiver with a known pattern for frame boundary detection.

Trellis Coding: Trellis coding is applied after frame sync encoding using a 16-state Binary Convolutional Code (BCC) with generator polynomials $G_1 = 25_8$ and $G_2 = 37_8$ [?]. The BCC operates at a punctured rate of $4/5$, converting the base rate $1/2$ encoder to produce 5 coded bits for every 4 input bits [?]. For 256-QAM, the overall trellis coded modulation rate is $19/20$ [?], producing 8-bit trellis-coded words (C-words) suitable for 256-QAM mapping. A differential precoder is incorporated to resolve phase ambiguity at the receiver [?].

256-QAM Symbol Mapping: Each 8-bit C-word is mapped to a complex IQ symbol on the 256-QAM constellation defined by J.83B Figure B.19 [?]. The output is a stream of complex IQ samples at the symbol rate of 5.360537 Msym/s [?]. The output is a stream of complex IQ samples at the symbol rate of 5.360537 Msym/s [?].

SRRC Pulse Shaping: The final stage applies SRRC pulse shaping to the symbol stream. Table B.3 of the standard specifies the frequency response as a SRRC filter with roll-off $\alpha \approx 0.12$ for 256-QAM [?]. The matched SRRC filter at the receiver completes the Nyquist pair, minimizing Inter-Symbol Interference (ISI) at the sampling instants [?].

3.3 Reed-Solomon Encoding

RS coding forms the outermost layer of the J.83B FEC stack, providing block-level error correction before the data enters the interleaver [?]. The same RS code is used for both 64-QAM and 256-QAM operation, though the FEC frame format differs between the two modulation types [?].

3.3.1 Code Definition

The J.83B RS encoder implements a systematic, extended (128, 122) RS code over the Galois field $\text{GF}(128) = \text{GF}(2^7)$ [?]. The code corrects up to $t = 3$ symbol errors per RS block [?]. Each symbol is 7 bits wide, matching the field size.

The field is constructed using the primitive polynomial [?]

$$p(x) = x^7 + x^3 + 1, \quad p(\alpha) = 0 \quad (3.1)$$

where α is the primitive element of $\text{GF}(128)$.

The generator polynomial is [?]

$$g(x) = (x - \alpha)(x - \alpha^2)(x - \alpha^3)(x - \alpha^4)(x - \alpha^5) \quad (3.2)$$

which expands to

$$g(x) = x^5 + \alpha^{52}x^4 + \alpha^{116}x^3 + \alpha^{119}x^2 + \alpha^{61}x + \alpha^{15} \quad (3.3)$$

3.3.2 Encoding Procedure

The encoder accepts a message block of 122 seven-bit symbols, represented as the message polynomial [?]

$$m(x) = m_{121}x^{121} + m_{120}x^{120} + \cdots + m_1x + m_0 \quad (3.4)$$

The message polynomial is multiplied by x^5 and divided by $g(x)$ to produce the remainder [?]

$$r(x) = r_4x^4 + r_3x^3 + r_2x^2 + r_1x + r_0 \quad (3.5)$$

These five coefficients constitute the five RS parity symbols. The parity symbols are appended to the message to form a 127-symbol code word [?].

$$c(x) = m_{121}x^{126} + \cdots + m_0x^5 + r_4x^4 + r_3x^3 + r_2x^2 + r_1x + r_0 \quad (3.6)$$

A valid code word has roots at α^1 through α^5 [?].

3.3.3 Extended Parity Symbol

The J.83B code is extended from (127, 121) to (128, 122) by evaluating the 127-symbol code word at the sixth power of α [?]

$$\bar{c} = c(\alpha^6) \quad (3.7)$$

This extended parity symbol $\bar{c}$ is appended as the final symbol of the transmitted RS block, yielding the complete 128-symbol output structure shown in Table 3.3.

Table 3.3. J.83B Reed-Solomon block structure [?].

Symbol positions	Content	Width
1 – 122	Information symbols ($m_{121} \dots m_0$)	7 bits each
123 – 127	RS parity symbols ($r_4 \dots r_0$)	7 bits each
128	Extended symbol ($\bar{c} = c(\alpha^6)$)	7 bits
Total	128 symbols	896 bits

The parameters of the RS code are summarized in Table 3.4.

Table 3.4. J.83B Reed-Solomon code parameters [?].

Parameter	Value
Code	(128, 122) extended RS
Field	$\text{GF}(2^7) = \text{GF}(128)$
Symbol width	7 bits
Information symbols	122 per block
Parity symbols	5 (RS) + 1 (extended) = 6 total
Output block size	128 symbols
Correction capability	$t = 3$ symbol errors per block

The RS encoder output feeds directly into the convolutional interleaver described in Section 3.4. The interleaver disperses the 128-symbol RS blocks across multiple frames so that burst channel errors are converted into correctable single-symbol errors at the RS decoder [?].

3.4 Convolutional Interleaving

Convolutional interleaving is applied to the RS-encoded byte stream after the encoder and before the randomizer [?]. Its purpose is to disperse burst channel errors across multiple

RS blocks so that a contiguous sequence of corrupted symbols at the receiver is spread into isolated single-symbol errors after de-interleaving, keeping the per-block error count within the three-symbol correction capability of the (128, 122) RS code [?].

3.4.1 Forney Interleaver Structure

The J.83B interleaver is a Forney convolutional type, operating on 7-bit symbols that match the RS field width [?]. The structure consists of I parallel delay branches, indexed $b = 0, 1, \dots, I - 1$, with a commutator that cycles through the branches sequentially at the RS symbol rate [?]. At the start of each FEC frame the commutator is initialized to branch 0 [?].

Branch b introduces a delay of [?]

$$d_b = b \cdot J \text{ symbols, } b = 0, 1, \dots, I - 1 \quad (3.8)$$

so that branch 0 passes symbols with zero delay, branch 1 delays by J symbols, branch 2 by $2J$ symbols, and so on up to branch $I - 1$ which introduces a delay of $(I - 1) \cdot J$ symbols. The de-interleaver at the receiver mirrors this structure, assigning to branch b a complementary delay of $(I - 1 - b) \cdot J$ symbols, so that the combined round-trip delay is constant at $(I - 1) \cdot J$ symbols for every symbol regardless of which branch it traverses [?].

The round-trip symbol latency through the interleaver–de-interleaver pair is therefore

$$L = I \cdot (I - 1) \cdot J \text{ symbols.} \quad (3.9)$$

3.4.2 Operating Levels and Mode Selection

J.83B defines two interleaver operating levels [?]. Level 1 supports a single fixed configuration ($I = 128$, $J = 1$) and is specified for DOCSIS 64-QAM transmission only [?]. Level 2

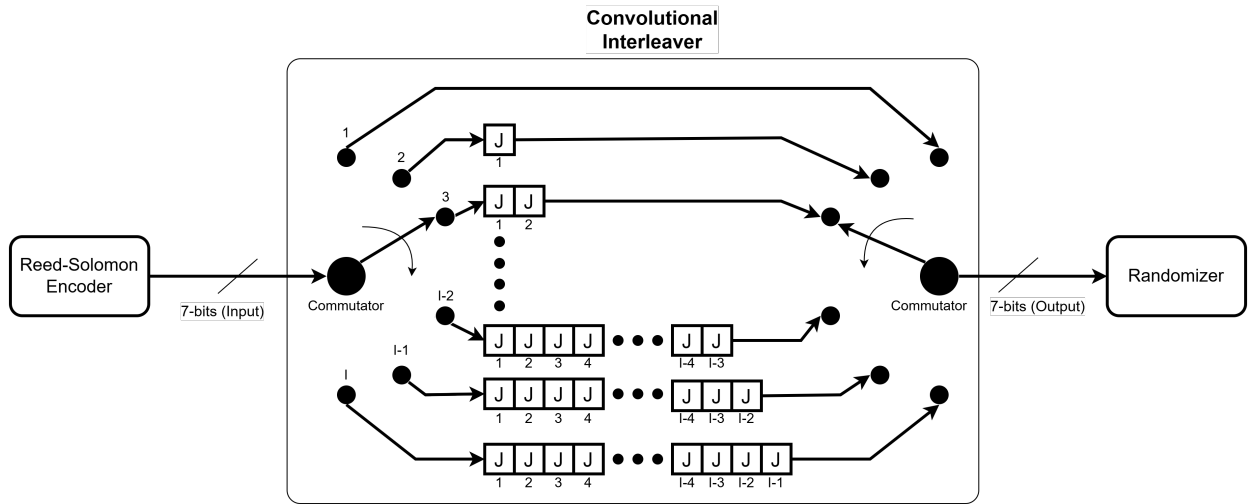


Figure 3.2. J.83B Annex B convolutional interleaver structure [?]. Branch b introduces a delay of $b \cdot J$ symbols; Branch 1 is a direct connection with zero delay. Input commutator cycles at the RS symbol rate; output feeds the randomizer.

extends interleaver support to both 64-QAM and 256-QAM modulation and allows variable interleaving parameters [?]. A 4-bit in-band control word transmitted during the FEC frame sync interval conveys the selected I and J values to the receiver for the active channel [?]. Table 3.5 lists the defined Level 2 parameter combinations together with the corresponding burst protection and latency figures from Table B.2 of the standard [?].

Table 3.5. J.83B Level 2 interleaver parameter options [?].					
Control Word	I	J	Burst Protection (μs)	Latency, 256-QAM (ms)	
0001	128	1	66		2.8
0011	64	2	33		1.4
0101	32	4	16		0.68
0111	16	8	8.2		0.33
1001	8	16	4.1		0.15

The burst protection and latency figures in Table 3.5 exhibit a design trade-off: deeper interleaving ($I = 128$, $J = 1$) provides greater immunity to sustained burst noise at the cost of higher end-to-end processing latency, while shallower configurations reduce latency at the expense of burst tolerance [?].

3.4.3 Parameters Used in This Work

This implementation operates in Level 2 mode with control word 0001, corresponding to $I = 128$ branches and increment $J = 1$ symbol per branch [?]. This is the maximum-depth configuration and provides the highest burst protection of the defined Level 2 modes [?]. Substituting into (3.9), the round-trip interleaver–de-interleaver latency is

$$L = 128 \cdot 127 \cdot 1 = 16,256 \quad \text{symbols}, \quad (3.10)$$

corresponding to a one-way 256-QAM channel latency of 2.8 ms as specified in Table B.2 of the standard [?]. The burst protection figure for this mode is $66 \mu\text{s}$ [?], which bounds the maximum contiguous burst duration that the combined interleaver and RS decoder can fully correct.

The interleaved symbol stream feeds directly into the randomizer described in Section 3.5, where a PN sequence is applied for spectral whitening prior to frame sync encoding and trellis coding [?].

3.5 Randomization

The randomizer is applied to the interleaved symbol stream after the convolutional interleaver and before frame sync encoding [?]. Its function is to ensure even distribution of symbols across the 256-QAM constellation, which is required for the demodulator to maintain carrier and timing lock [?]. Without randomization, long runs of identical symbols produce spectral lines in the transmitted signal and reduce the effectiveness of the automatic gain control and phase-locked loop circuits in the receiver [?].

3.5.1 PN Sequence Generator

The randomizer XORs each 7-bit interleaved symbol with a corresponding symbol from a PN sequence defined over $\text{GF}(128) = \text{GF}(2^7)$ [?]. The PN sequence is generated by a Linear Feedback Shift Register (LFSR) governed by the $\text{GF}(128)$ recurrence polynomial [?]

$$f(x) = x^3 + x + \alpha^3 \quad (3.11)$$

where α is the primitive element of $\text{GF}(128)$ satisfying the same reduction polynomial used by the RS encoder [?],

$$p(x) = x^7 + x^3 + 1. \quad (3.12)$$

The LFSR consists of three 7-bit state registers operating over $\text{GF}(128)$ [?]. At each clock step the register state advances according to the recurrence in (3.11), producing one 7-bit PN output symbol per step. Each output symbol is bitwise XOR-ed with the corresponding interleaved data symbol, leaving the 7-bit width of the symbol stream unchanged [?].

3.5.2 Initialization and Frame Synchronization

The randomizer is initialized at each FEC frame boundary [?]. Initialization loads all three LFSR state registers to the all-ones state `0x7F` [?]. The initialization occurs during the FEC frame sync trailer interval; the trailer symbols themselves are therefore not randomized [?]. The initialized register state produces the first PN output symbol, which randomizes the first data symbol of the subsequent FEC frame [?].

For 256-QAM operation, one FEC frame consists of 88 RS blocks of 128 symbols each, giving a frame data payload of $88 \times 128 = 11,264$ symbols [?]. The PN sequence therefore has a period of 11,264 symbols per frame and is re-initialized at the start of each frame [?].

3.5.3 Randomizer Parameters

The randomizer parameters for 256-QAM operation are summarized in Table 3.6.

Table 3.6. J.83B Annex B randomizer parameters for 256-QAM operation [?].

Parameter	Value
Sequence type	LFSR over GF(2^7)
Recurrence polynomial	$f(x) = x^3 + x + \alpha^3$
Reduction polynomial	$p(x) = x^7 + x^3 + 1$
Symbol width	7 bits
Initialization state	0x7F (all ones)
Init trigger	FEC frame trailer interval
Operation	Bitwise XOR per symbol
Sequence period	$88 \times 128 = 11,264$ symbols

The randomized symbol stream feeds directly into the frame sync encoder described in Section 3.6, where the FEC frame trailer is appended and the symbol stream is unpacked into a serial bit stream for trellis coding [?].

3.6 Frame Sync Encoding

Frame sync encoding appends a fixed-pattern trailer to the randomized symbol stream at the end of each FEC frame [?]. The trailer serves two functions at the receiver: it delineates the FEC frame boundary, enabling synchronized RS decoding, de-interleaving, and de-randomization; and it carries the 4-bit in-band control word that conveys the active interleaver depth mode to the receiver [?]. For 256-QAM, trellis group boundaries are additionally aligned to the FEC frame by the frame sync encoder [?].

3.6.1 FEC Frame Structure

For 256-QAM operation, one FEC frame consists of 88 RS blocks each containing 128 seven-bit symbols [?]. Each 7-bit symbol is serialized Most Significant Bit (MSB)-first into a contiguous bit stream, producing [?]

$$N_{\text{data}} = 88 \times 128 \times 7 = 78,848 \text{ bits} \quad (3.13)$$

of data per frame. The 40-bit sync trailer is appended at the end of the data bit stream, giving a total frame length of [?]

$$N_{\text{frame}} = 78,848 + 40 = 78,888 \text{ bits} \quad (3.14)$$

which divides exactly into 2076×38 bits, corresponding to 2076 trellis groups of 38 input bits each [?]. The RS block and 7-bit symbol boundaries are aligned to the end of the FEC frame [?].

Table 3.7. J.83B Annex B 256-QAM FEC frame arithmetic [?].

Quantity	Value
RS blocks per frame	88
Symbols per RS block	128
Bits per symbol (serialized)	7
Data bits per frame	78,848
Sync trailer bits	40
Total bits per frame	78,888
Trellis groups per frame	2,076
Bits per trellis group	38

3.6.2 Sync Trailer Format

The 40-bit sync trailer is structured as three fields packed MSB-first [?], as shown in Table 3.8.

Table 3.8. J.83B Annex B 256-QAM FEC frame sync trailer structure [?].

Field	Content	Width	Value
Unique word	Fixed synchronization pattern	32 bits	71 E8 4D D4 _{HEX}
Control word	Interleaver depth mode	4 bits	Level 2 mode code
Reserved	Set to zero	4 bits	0x0
Total	Sync trailer	40 bits	—

The 32-bit unique word 71 E8 4D D4_{HEX} is the fixed binary pattern

0111 0001 1110 1000

0100 1101 1101 0100

[?]. The receiver searches for this pattern to detect the FEC frame boundary [?]. The 4-bit control word immediately follows and encodes the active Level 2 interleaver depth mode as defined in Table B.2 of the standard [?]. The final 4 bits are reserved and set to zero [?].

The sync trailer is inserted by the encoder and is not randomized [?]. The randomizer, described in Section 3.5, is initialized during the trailer interval so that its output is ready to scramble the first data symbol of the following frame [?].

3.6.3 Symbol Serialization

Prior to trellis coding, the frame sync encoder serializes each 7-bit RS symbol into individual bits, emitting the most significant bit first [?]. The most significant bit of the first RS symbol of each FEC frame is assigned to the first bit position of the first non-sync trellis group [?]. There is no synchronization relationship between the RS block boundaries and MPEG-TS packet boundaries; MPEG-TS packet synchronization is obtained independently from FEC frame synchronization, keeping the FEC and transport layers decoupled [?].

The serialized bit stream, comprising the 78,848 data bits and the 40-bit trailer (Table 3.7), feeds directly into the trellis encoder described in Section 3.7 [?].

3.7 Trellis Coding

Trellis coding forms the inner code of the J.83B concatenated coding scheme [?]. It introduces redundancy into the transmitted symbol stream to improve the threshold Signal-to-Noise Ratio (SNR) by expanding the symbol constellation without increasing the symbol rate, and is more precisely termed trellis coded modulation [?]. For 256-QAM, the trellis encoder

accepts the serialized bit stream from the frame sync encoder and produces 8-bit trellis-coded words, referred to as C-words, that address the 256-QAM constellation mapper [?].

3.7.1 Trellis Group Structure

The trellis encoder operates on fixed-length groups of input bits called trellis groups [?]. For 256-QAM, each trellis group contains 38 input bits and produces 5 output 256-QAM symbols, for a total of 40 output bits per group [?].

Two types of trellis groups are defined for 256-QAM [?]:

- **Non-sync group:** contains 38 data bits, all sourced from the serialized RS symbol stream [?].
- **Sync group:** contains 30 data bits and 8 sync bits drawn from the FEC frame sync trailer [?]. The sync bits occupy only the convolutionally encoded bit positions within the group [?].

Since each FEC frame contains 78,888 bits total (88 RS blocks plus the 40-bit trailer), the frame divides into $78,888/38 = 2,076$ trellis groups [?]. Of these, 2,071 are non-sync groups and 5 are sync groups; the 5 sync groups appear at the end of the frame [?].

Within each trellis group, the 38 input bits are partitioned by a data formatter into four streams: the MSBs of the ‘A’ symbols, the MSBs of the ‘B’ symbols, the Least Significant Bit (LSB)s of the ‘A’ symbols, and the LSBs of the ‘B’ symbols [?]. The MSB streams bypass the convolutional encoder and are passed directly to the 256-QAM mapper uncoded [?]. The LSB streams are differentially precoded and then convolutionally encoded before mapping [?].

3.7.2 Binary Convolutional Coder

The convolutional encoder is a 16-state, non-systematic, rate $\frac{1}{2}$ BCC with generator polynomials [?]]

$$G_1(D) = 1 + D^2 + D^4 \quad (3.15)$$

$$G_2(D) = 1 + D + D^2 + D^3 + D^4 \quad (3.16)$$

corresponding to the octal pair $(G_1, G_2) = (25_8, 37_8)$ [?]]. For each input bit presented to the encoder, two bits are produced at the output: the G_1 output followed by the G_2 output [?]]. The commutator is initialized to the G_1 position at the beginning of each trellis group [?]].

Four input bits per trellis group produce 8 convolutionally encoded bits [?]]. These 8 bits are reduced to 5 by a puncture matrix applied over the 4-input block [?]]

$$\begin{bmatrix} P_1 \\ P_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \quad (3.17)$$

where 1 denotes transmission and 0 denotes suppression. The puncture matrix converts the rate $\frac{1}{2}$ encoder to rate $\frac{4}{5}$, retaining 5 of the 8 encoded bits per trellis group [?]]. Two independent BCC instances operate in parallel: one encodes the ‘A’ LSB stream producing outputs U_1 through U_5 , and one encodes the ‘B’ LSB stream producing outputs V_1 through V_5 [?]].

3.7.3 Differential Precoder

Prior to convolutional encoding, the LSB streams pass through a differential precoder that provides 90° rotational invariance [?]]. Rotational invariance allows the receiver to recover data correctly following a carrier phase slip without requiring FEC resynchronization, since

information is carried by the change in phase rather than its absolute value [?]. For 256-QAM, the 4th and 8th bits of each C-word are differentially encoded, corresponding to positions C_4 and C_0 in the constellation labelling of Figure B.19 of the standard [?].

The differential precoder update equations are [?]

$$x_J = W_J \oplus x_{J-1} \oplus Z_J \cdot (x_{J-1} \oplus Y_{J-1}) \quad (3.18)$$

$$Y_J = Z_J \oplus W_J \oplus Y_{J-1} \oplus Z_J \cdot (x_{J-1} \oplus Y_{J-1}) \quad (3.19)$$

where W_J and Z_J are the two input bits at step J , x_J and Y_J are the two precoded output bits, $\oplus$ denotes XOR, and $(\cdot)$ denotes the AND operation [?].

3.7.4 Overall Code Rate and Output

Each trellis group produces 40 output bits from 38 input bits, giving an overall trellis coded modulation rate of [?]

$$R_{\text{TCM}} = \frac{38}{40} = \frac{19}{20} \quad (3.20)$$

The 40 output bits per trellis group are packed into 5 eight-bit C-words, each of which addresses one symbol position in the 256-QAM constellation [?]. The trellis encoder output feeds directly into the 256-QAM symbol mapper described in Section 3.8 [?].

The trellis coding parameters for 256-QAM operation are summarized in Table 3.9.

3.8 256-QAM Symbol Mapping

The 256-QAM symbol mapper converts the 8-bit C-words produced by the trellis encoder into complex IQ baseband symbols for transmission [?]. Each C-word addresses a look-up table that returns the complex symbol coordinates corresponding to that C-word's position

Table 3.9. J.83B Annex B trellis coding parameters for 256-QAM operation [?].

Parameter	Value
Encoder type	16-state non-systematic BCC
Generator polynomials	$G_1 = 25_8, G_2 = 37_8$
Base code rate	1/2
Puncture matrix	$[P_1; P_2] = [0001; 1111]$
Punctured code rate	4/5
Overall Trellis Coded Modulation (TCM) rate	19/20
Input bits per group	38
Output symbols per group	5
Output bits per group	40
Trellis groups per frame	2,076 (2,071 non-sync + 5 sync)
C-word width	8 bits

on the 256-QAM constellation defined by Figure B.19 of the standard [?]. The mapper output is a stream of complex IQ samples at the J.83B symbol rate of 5.360537 Msym/s, which feeds directly into the SRRC pulse shaping filter described in Section 3.9 [?].

3.8.1 Constellation Geometry

The J.83B 256-QAM constellation is a square, 256-point array arranged on a 16×16 grid in the IQ plane [?]. The in-phase and quadrature axes each carry 16 equally spaced amplitude levels at odd integer values $\{-15, -13, \dots, -1, +1, \dots, +13, +15\}$, giving $16^2 = 256$ distinct symbol positions [?]. The constellation is symmetric about both the I and Q axes, and the minimum Euclidean distance between adjacent symbols is uniform across the grid [?].

3.8.2 C-Word to Symbol Mapping

The trellis encoder delivers the coded and uncoded 4-bit ‘A’ and ‘B’ streams to the mapper, which concatenates them into a single 8-bit C-word with C-bit ordering $C_7C_6C_5C_4C_3C_2C_1C_0$ where C_7 is the MSB [?]. The 8-bit C-word takes values in the range 0–255 and uniquely identifies one of the 256 constellation points via the look-up table defined by Figure B.19 of

the standard [?]. The mapping is a bijection: every C-word value corresponds to exactly one symbol position, and every symbol position is addressed by exactly one C-word [?].

The differential precoder bits C_4 and C_0 occupy the two differentially encoded positions within the C-word, as noted in Section 3.7 [?]. The remaining six C-word bits are passed through uncoded and carry the MSB streams directly from the data formatter [?].

3.8.3 IQ Output and Symbol Rate

The mapper output is a stream of complex baseband samples of the form $s_k = I_k + jQ_k$, where I_k and Q_k are the in-phase and quadrature coordinates of the k -th mapped symbol [?]. One complex sample is produced per C-word, at a rate equal to the J.83B symbol rate of 5.360537 Msym/s [?].

The 256-QAM symbol mapping parameters are summarized in Table 3.10.

Table 3.10. J.83B Annex B 256-QAM symbol mapping parameters [?].

Parameter	Value
Constellation	Square 256-QAM
Grid size	16×16
Symbol positions	256
I/Q amplitude levels	$\{-15, -13, \dots, +13, +15\}$
Input C-word width	8 bits ($C_7 - C_0$)
Mapping definition	Figure B.19, ITU-T J.83 Annex B
Implementation	Look-up table (bijection)
Output	Complex IQ symbol stream
Symbol rate	5.360537 Msym/s

3.9 SRRC Pulse Shaping

SRRC pulse shaping is the final stage of the J.83B transmit processing chain, applied to the complex IQ symbol stream after 256-QAM mapping [?]. The SRRC filter bandlimits the symbol stream to fit within the 6 MHz channel allocation, controls the spectral shape of the transmitted signal, and forms one half of the matched filter pair that minimizes ISI at

the receiver sampling instants [?]. The standard refers to this filter as a square-root raised cosine filter in Table B.3 and as a Root-Nyquist filter in §B.6.2 [?].

3.9.1 Nyquist Matched Filter Pair

The J.83B pulse shaping scheme employs a matched SRRC filter pair: an SRRC filter at the transmitter and an identical SRRC filter at the receiver [?]. The cascade of the transmit and receive SRRC filters satisfies the Nyquist ISI criterion [?], producing a raised cosine overall frequency response

$$H_{\text{RC}}(f) = H_{\text{TX}}(f) \cdot H_{\text{RX}}(f) = |H_{\text{SRRC}}(f)|^2 \quad (3.21)$$

where $H_{\text{TX}}(f) = H_{\text{RX}}(f) = H_{\text{SRRC}}(f)$ are the identical transmit and receive SRRC frequency responses [?]. The raised cosine response $H_{\text{RC}}(f)$ has zero crossings at all non-zero integer multiples of the symbol period $T_s = 1/f_{\text{sym}}$, ensuring zero ISI at the receiver sampling instants when the channel is ideal [?].

The SRRC frequency response is [?]

$$H_{\text{SRRC}}(f) = \begin{cases} \sqrt{T_s}, & |f| \leq \frac{1-\alpha}{2T_s} \\ \sqrt{\frac{T_s}{2} \left[1 + \cos\left(\frac{\pi T_s}{\alpha} \left(|f| - \frac{1-\alpha}{2T_s} \right) \right) \right]}, & \frac{1-\alpha}{2T_s} < |f| \leq \frac{1+\alpha}{2T_s} \\ 0, & |f| > \frac{1+\alpha}{2T_s} \end{cases} \quad (3.22)$$

where $f_{\text{sym}} = 1/T_s$ is the symbol rate and α is the roll-off factor [?]. The one-sided occupied bandwidth of the filtered signal is $(1+\alpha)/(2T_s)$, so the total double-sided bandwidth is $(1+\alpha) \cdot f_{\text{sym}}$ [?].

3.9.2 J.83B Filter Specification

For 256-QAM, Table B.3 of the standard specifies the transmit filter as a square-root raised cosine with roll-off factor [?]

$$\alpha = 0.12 \quad (256\text{-QAM}) \quad (3.23)$$

and the symbol rate [?]

$$f_{\text{sym}} = 5.360537 \text{ Msym/s} \pm 5 \text{ ppm}. \quad (3.24)$$

Substituting into the bandwidth expression, the double-sided occupied bandwidth of the J.83B 256-QAM channel is [?]

$$B = (1 + \alpha) \cdot f_{\text{sym}} = 1.12 \times 5.360537 \approx 6.004 \text{ MHz} \quad (3.25)$$

which fits within the 6 MHz channel slot defined by the North American channel plan [? ?].

3.9.3 RF Output Signal Model

The RF modulated output signal is formed by modulating the SRRC-filtered baseband IQ components onto the carrier frequency f_c [?]

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t) \quad (3.26)$$

where $I(t)$ and $Q(t)$ are the Root-Nyquist filtered baseband in-phase and quadrature components of the constellation symbols, and f_c is the RF carrier frequency [?].

The J.83B SRRC pulse shaping parameters are summarized in Table 3.11. Filter implementation parameters - including filter span, samples per symbol, tap count, and output sample rate - are implementation-specific design choices and are described in Chapter 4.

Table 3.11. J.83B Annex B SRRC pulse shaping parameters for 256-QAM operation (standard-sourced) [?].

Parameter	Value
Filter type	Square-root raised cosine (SRRC)
Standard reference	§B.6.1 Table B.3, §B.6.2
Roll-off factor α	0.12 (256-QAM)
Symbol rate input	5.360537 Msym/s $\pm$ 5 ppm
Occupied bandwidth	$\approx$ 6.004 MHz
Channel slot	6 MHz
Matched filter	TX SRRC $\times$ RX SRRC = RC
ISI at sampling	Zero (Nyquist criterion, ideal channel)

CHAPTER 4

IMPLEMENTATION

4.1 System Architecture Overview

4.2 Processing System

4.2.1 UDP/GigE RX

4.2.2 TX Chain

4.2.3 DMA Control

4.3 Programmable Logic: TX Chain

4.3.1 PL-01: AXI DMA MM2S

4.3.2 PL-02: Transport Framing

4.3.3 PL-03: Reed-Solomon Encoder

4.3.4 PL-04: Convolutional Interleaver

4.3.5 PL-05: Randomizer

4.3.6 PL-06: Frame Sync Encoder

4.3.7 PL-07: Trellis Encoder

4.3.8 PL-08: QAM-256 Mapper

4.3.9 PL-09: SRRC FIR TX

4.3.10 PL-10: SD-FEC LDPC

4.3.11 PL-11: Register File RTL

4.3.12 PL-12: IQ Combiner

4.4 Programmable Logic: RX Chain

4.4.1 PL-13: SRRC FIR RX

4.4.2 PL-14: Symbol Timing Recovery

4.4.3 PL-15: Carrier and Phase Recovery

4.4.4 PL-16: QAM-256 Demapper

4.4.5 PL-17: Trellis Decoder

4.4.6 PL-18: Frame Sync Detection

4.4.7 PL-19: De-Randomizer

4.4.8 PL-20: De-Interleaver

4.4.9 PL-21: Reed-Solomon Decoder

4.4.10 PL-22: Transport Framing RX

4.4.11 PL-23: AXI DMA S2MM

4.5 RF Data Converter

4.5.1 DAC: Tile 228

4.5.2 ADC: Tile 224

4.6 RF Front-End

4.6.1 TX Path

4.6.2 RX Path

4.7 Clocking Subsystem

4.7.1 LMK04828

4.7.2 LMX2594

4.7.3 xrfclk API

4.8 Test and Measurement

4.8.1 H-Thunder-1 Reference Signal

4.8.2 HDHomeRun: Source and Sink

4.8.3 Spectrum Analysis

4.9 Verification and Results

4.9.1 Test Vectors

4.9.2 Loopback Validation

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

5.2 Future Work

BIOGRAPHICAL SKETCH

Sathwik Sudhindranath received the B.E. degree in Electronics and Communication Engineering from Visvesvaraya Technological University, India, in 2021. He is currently pursuing the M.S. degree in Electrical Engineering with a concentration in Computing Systems at The University of Texas at Dallas, Richardson, Texas, expected December 2026.

CURRICULUM VITAE